Figure 1.1

KING KHALID UNIVERSITY

COLLEGE OF COMPUTER SCIENCE

Deep Learning-Based Depression Detection from Brain MRI

By

Name	Academic Number
Fahad Abdullah Saeed Al-Nahi Alqahtani	**444801561**
Abdullah Khalid Abdullah Alshaharani	**444801741**
Ibrahim Yahya Ali Hurubi	**444809883**
Fahad Awad Yahya Al-Mubarak Asiri	**444801613**
Ali Ahmed Yahya Muyini	**444803140**

Supervised By :

Dr. Mohamed Ghouse

1447 - 2026

COMMITTEE APPROVAL REPORT

We certify that we have read this graduation project report as examining committee, examined the student in its content and that in our opinion it is adequate as a project document for B.Sc. in Computer Science.

Committee Member	**Position**	**Signature**	**Date**
Dr. _____	Supervisor	_____	_____
Dr. _____	Examiner 1	_____	_____
Dr. _____	Examiner 2	_____	_____

Final Decision:

☐ Approved ☐ Approved with Minor Corrections ☐ Rejected

DEDICATION

In the name of Allah, the Most Gracious, the Most Merciful.

We dedicate this work to our beloved families, whose encouragement has been our greatest strength.

To our parents, for their endless sacrifices, prayers, and belief in our abilities—every achievement in our lives is because of you.

To our brothers and sisters, thank you for your support, motivation, and joy.

This work is also dedicated to everyone striving to improve mental-health diagnosis and care, and to every student who continues to push forward despite challenges.

May this project be a small contribution toward a better future.

ACKNOWLEDGMENT

First and foremost, all praise is due to Allah for granting us the strength, patience, and ability to complete this project.

We would like to extend our deepest gratitude to our supervisor,

Dr. Mohamed Ghouse, for his guidance, continuous support, and valuable insights that shaped the direction of this research. His expertise in AI and biomedical imaging played a crucial role in improving the quality and depth of this work.

Special thanks to the Computer Science Department at King Khalid University for providing the academic environment, resources, and technical support essential to completing this project.

We also acknowledge The Depression Imaging REsearch ConsorTium (DIRECT) and the R-fMRI Lab for providing the neuroimaging datasets used in this study, including the DIRECT Phase II dataset and the REST-meta-MDD dataset. These datasets served as the foundation for developing and evaluating the proposed deep learning model.

To our families—thank you for your love, patience, prayers, and constant encouragement throughout our academic journey. Your belief in us has made this achievement possible.

Finally, we would like to express our appreciation to our friends and colleagues who provided help, inspiration, and motivation during difficult stages of this project.

Thank you all.

ABSTRACT

Major Depressive Disorder (MDD) is a serious mental health condition that affects millions of people worldwide and remains difficult to diagnose objectively using only clinical interviews and self-report questionnaires. Traditional diagnosis depends heavily on subjective assessment, which may lead to delayed or inconsistent detection. Structural Magnetic Resonance Imaging (sMRI) provides a promising direction for objective analysis because it captures three-dimensional anatomical information related to brain regions associated with depression, such as the hippocampus, amygdala, prefrontal cortex, and anterior cingulate cortex.

This project presents an AI-assisted framework for detecting Major Depressive Disorder from T1-weighted 3D brain MRI scans and clinical metadata. The proposed system uses a State-of-the-Art-inspired multimodal late-fusion deep learning approach. The MRI branch extracts anatomical features from full 3D brain volumes using a 3D convolutional neural network enhanced with Squeeze-and-Excitation attention, while the clinical branch processes patient-level metadata such as sex, age, education, and HAMD score using a Multi-Layer Perceptron. A 3D convolutional autoencoder is first pretrained on Healthy Control MRI scans to learn a normative representation of healthy brain anatomy. The learned encoder is then used as an anatomical blueprint for the final diagnostic model.

The system was implemented as a research prototype consisting of a Next.js and React frontend, a FastAPI backend, and a PyTorch-based multimodal deep learning model. The interface allows users to upload or select an MRI scan, enter clinical metadata, run AI analysis, and view the predicted class, confidence score, MRI preview, and Grad-CAM-style heatmap visualization. The backend receives the MRI-related input and metadata, performs inference using the trained model, and returns the diagnostic output to the frontend.

The final model was evaluated on an independent test set of 217 samples. The results showed strong research performance, achieving 97% accuracy, 0.9870 AUC, 0.9440 sensitivity/recall, and 1.0000 specificity. The confusion matrix showed 92 correctly classified Healthy Control subjects, 118 correctly classified MDD subjects, and 7 false negative cases. These results demonstrate that the proposed framework is promising as an academic AI-assisted decision-support prototype. However, the system is not a certified clinical diagnostic tool and requires further external validation, clinical expert review, privacy improvements, and secure deployment before real clinical use.

Keywords: Major Depressive Disorder, Brain MRI, Deep Learning, 3D CNN, Multimodal Learning, Grad-CAM, FastAPI, Clinical Decision Support.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

Abbreviation / Symbol	Meaning
ACC	Anterior Cingulate Cortex
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
BCE	Binary Cross Entropy
BIDS	Brain Imaging Data Structure
CNN	Convolutional Neural Network
CT	Computed Tomography
DFD	Data Flow Diagram
DL	Deep Learning
DNN	Deep Neural Network
EEG	Electroencephalography
FN	False Negative
FP	False Positive
fMRI	Functional Magnetic Resonance Imaging
FR	Functional Requirement
Grad-CAM	Gradient-weighted Class Activation Mapping
HAMD / HAM-D	Hamilton Depression Rating Scale
HC	Healthy Control

Abbreviation / Symbol	Meaning
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
MDD	Major Depressive Disorder
ML	Machine Learning
MLP	Multi-Layer Perceptron
MRI	Magnetic Resonance Imaging
MSE	Mean Squared Error
NFR	Non-Functional Requirement
NIfTI	Neuroimaging Informatics Technology Initiative
PET	Positron Emission Tomography
PFC	Prefrontal Cortex
PHQ-9	Patient Health Questionnaire-9
REST-meta-MDD	Resting-State fMRI Data Sharing Project for MDD
ROC	Receiver Operating Characteristic
SE	Squeeze-and-Excitation
SHAP	SHapley Additive exPlanations
sMRI	Structural Magnetic Resonance Imaging
SOTA	State-of-the-Art
SVM	Support Vector Machine
TN	True Negative
TP	True Positive

Abbreviation / Symbol	Meaning
UI	User Interface
VBM	Voxel-Based Morphometry
XAI	Explainable Artificial Intelligence
3D	Three-Dimensional
.npy	NumPy array file format
.nii / .nii.gz	NIfTI medical imaging file format
(TP)	Correctly predicted MDD case
(TN)	Correctly predicted Healthy Control case
(FP)	Healthy Control case incorrectly predicted as MDD
(FN)	MDD case incorrectly predicted as Healthy Control
Accuracy	Percentage of all correct predictions
Sensitivity / Recall	Ability of the model to correctly detect MDD cases
Specificity	Ability of the model to correctly detect Healthy Control cases
F1-score	Harmonic mean of precision and recall
Late Fusion	Combining MRI and clinical features after separate feature extraction
Confidence Score	Probability-based score indicating model prediction confidence

CHAPTER 1: INTRODUCTION

1.1 Background

Major Depressive Disorder (MDD) is a top cause of disability worldwide, with over 280 million people across the world being impacted [1]. In Saudi Arabia, as in the rest of the world, diagnosis of depression remains a significant challenge. Clinicians currently primarily rely on patient self-reporting questionnaires (e.g., the PHQ-9) and subjective clinical interviews. Although important, these are prone to being biased and may not capture the full neurobiological basis of the disorder.

This project completes this gap by adopting an objective, quantitative perspective: that of structural Magnetic Resonance Imaging (sMRI). Unlike functional MRI (fMRI), which measures activity in the brain, sMRI provides a 3D high-resolution picture of the brain's anatomy. A significant amount of neuroscience literature has determined that MDD has been linked to quantifiable, albeit small, morphological abnormalities [2]. These are:

- Volumetric Changes: Most notably, reduced volume of the amygdala and hippocampus [2].
- Cortical Thinning: Loss of gray matter thickness in critical regions like the prefrontal cortex and the anterior cingulate cortex (ACC) [2].

Although these structural changes are noted, they are typically too small or spatially complex for the naked human eye to detect individually. Artificial intelligence and deep learning provide a promising direction for supporting MRI-based depression detection by learning complex structural patterns from brain images [3], [4]. Therefore, this project focuses on developing a web-based AI-assisted prototype that allows users to upload brain MRI scans and view diagnostic insights through an interactive interface.

1.2 Problem Statement

Current psychiatric diagnosis of depression lacks quantitative, reproducible imaging biomarkers. The reliance on subjective interpretation and self-reporting implies there is delayed diagnosis, inconsistent detection, and a lack of ability to distinguish MDD from other disorders.

Furthermore, while sMRI data is widely available, radiologists face significant challenges in manually analyzing high-dimensional 3D scans for subtle morphological patterns. A manual, slice-by-slice inspection is time-consuming and may miss distributed biomarkers that are only visible when analyzing the brain volume as a whole.

Therefore, there is a pressing need for an AI-assisted diagnostic system that can:

1. Automatically and rapidly process 3D-sMRI scans.

2. Accurately identify complex structural patterns associated with depression.
3. Provide objective, quantitative evidence to support a clinician's diagnosis.

This project aims to build such a system, bridging the gap between advanced computational imaging and practical clinical psychiatry.

1.3 Proposed Solution

To address the problem of subjective and non-quantitative depression diagnosis, this project proposes the design, implementation, and evaluation of an AI-assisted diagnostic framework for Major Depressive Disorder detection.

The proposed solution is a SOTA-inspired multimodal deep learning system that analyzes T1-weighted 3D structural MRI scans together with selected clinical metadata [5], [6]. The imaging branch extracts anatomical features from 3D MRI volumes using a convolutional encoder enhanced with Squeeze-and-Excitation attention, while the clinical branch processes metadata such as age, sex, education, and HAMD score using a Multi-Layer Perceptron.

Before final classification, a 3D convolutional autoencoder is pretrained using Healthy Control MRI scans to learn a normative representation of healthy brain anatomy. The learned encoder is then used as an anatomical blueprint for the final diagnostic model. The MRI and clinical representations are combined through a late-fusion strategy and passed into a binary classification head to distinguish between Healthy Control and MDD subjects.

To improve clinical trust and interpretability, the framework is designed to support Explainable AI techniques such as Grad-CAM, which can highlight anatomical regions that contribute most strongly to the model’s prediction [4].

1.4 Research Questions

This project seeks to answer the following key questions:

1. Can a multimodal deep learning model combining 3D structural MRI scans and clinical metadata accurately distinguish between Major Depressive Disorder and Healthy Control subjects?
2. Does integrating clinical metadata with MRI-based anatomical features improve the diagnostic performance of MDD detection?
3. Can unsupervised autoencoder pretraining on Healthy Control MRI scans provide a useful normative anatomical representation for downstream MDD classification?
4. Can Explainable AI techniques such as Grad-CAM provide interpretable visual evidence by highlighting brain regions that influence the model’s prediction?

1.5 Objectives

To answer the research questions, the following objectives will be met:

1. To develop a multimodal AI-assisted framework for binary classification of Major Depressive Disorder and Healthy Control subjects.
2. To preprocess T1-weighted 3D structural MRI scans and integrate them with clinical metadata, including age, sex, education, and HAMD score.
3. To pretrain a 3D convolutional autoencoder on Healthy Control MRI scans to learn a normative representation of healthy brain anatomy.
4. To design and train a SOTA-inspired multimodal late-fusion model combining a 3D MRI imaging branch, Squeeze-and-Excitation attention, and a clinical metadata MLP branch.
5. To evaluate the model using accuracy, precision, recall/sensitivity, specificity, F1-score, confusion matrix, ROC curve, and AUC.
6. To support model interpretability using Explainable AI techniques such as Grad-CAM.
7. To develop a proof-of-concept prototype interface for displaying MRI input, prediction results, confidence score, and explainability outputs.

1.6 Scope of Work

This project will operate within the following defined boundaries:

- In Scope:
  - Data: T1-weighted structural MRI scans for MDD and Healthy Control subjects.
  - Clinical Metadata: Age, sex, education, and HAMD score.
  - Models: 3D convolutional neural networks, autoencoder-based healthy-brain pretraining, Squeeze-and-Excitation attention, clinical MLP branch, and multimodal late-fusion classification.
  - Analysis: 3D volumetric MRI analysis and multimodal MRI-clinical classification.
  - Explainability: Grad-CAM-based visual explanation for MRI regions contributing to model predictions.
  - Prototype: A proof-of-concept interface for uploading MRI scans and displaying prediction results.

- Out of Scope:
  - Other imaging modalities such as fMRI, resting-state fMRI, EEG, PET, or CT.
  - Non-imaging behavioral data such as speech, facial emotion, or text.
  - Other psychiatric or neurological disorders.
  - Real-time clinical deployment or certified medical diagnosis.
  - Treatment recommendation or medication planning.

- Implementation Tools: Next.js, React, TypeScript, Tailwind CSS, and

Vercel for developing and deploying the web-based prototype interface. Python and deep learning frameworks such as TensorFlow/PyTorch are considered for future backend AI model integration.

- Target Users: The conceptual prototype is designed for psychiatrists, radiologists, mental health researchers, and academic evaluators who need an interactive interface to demonstrate AI-assisted depression detection from brain MRI scans.

1.7 Expected Impact

The successful completion of this project may contribute to Saudi mental health research and clinical decision-support by providing an AI-assisted framework for objective MDD screening using brain MRI and clinical metadata.

- For Clinicians: The site will provide radiologists and psychiatrists with an objective, AI-driven screening tool. Numerical predictions and visual Grad-CAM heatmaps will serve as a useful 'second opinion,' presenting data-driven information about a patient's brain structure to supplement and aid in conventional clinical assessments.
- For Patients: By offering a basis for earlier and more objective identification, this instrument can reduce diagnostic delays. This allows patients to receive the early treatment they need, improving long-term health outcomes and reducing the individual and societal expense of untreated depression.
- For Research: The project will contribute a validated 3D deep learning model for sMRI analysis to King Khalid University's research and innovation ecosystem. It lays the foundation for possible research in the future, for example, transfer of the model to local Saudi datasets.
- For National Objectives: By developing an innovative digital-health solution to a high-priority public health problem, this project is directly aligned with the Saudi Vision 2030 healthcare transformation and innovation goals [11].

1.8 Thesis Organization

This thesis is organized into eight chapters and one supporting notebook section. Each chapter presents a specific part of the proposed AI-assisted depression detection framework using brain MRI and clinical metadata.

Chapter 1: Introduction
This chapter introduces the background of Major Depressive Disorder (MDD), the limitations of current diagnostic methods, and the motivation for using structural MRI and deep learning. It also presents the problem statement, proposed solution, research questions, objectives, scope of work, expected impact, and thesis organization.

Chapter 2: Review of Literature
This chapter reviews previous studies related to MRI-based depression detection. It discusses structural MRI biomarkers, traditional machine learning approaches, deep learning methods, multimodal learning, and Explainable AI. It also identifies the research gaps that this project aims to address.

Chapter 3: Methodology and Analysis
This chapter explains the methodology followed in the project. It covers dataset acquisition, MRI preprocessing, clinical metadata preparation, dataset splitting, model development, training strategy, evaluation metrics, and explainability support.

Chapter 4: System Design
This chapter presents the design of the proposed system. It includes system requirements, functional and non-functional requirements, system models, use case descriptions, data flow diagrams, system architecture, data organization, and interface design.

Chapter 5: System Implementation
This chapter describes how the system was implemented as a working prototype. It explains the frontend implementation using Next.js, React, TypeScript, and Tailwind CSS, the backend implementation using FastAPI, the AI model integration, ngrok testing, and the workflow between the interface and backend.

Chapter 6: Development of the Multimodal MDD Detection Model
This chapter focuses specifically on the deep learning model. It explains the multimodal dataset engine, 3D autoencoder pretraining, MRI imaging branch, Squeeze-and-Excitation attention, clinical metadata branch, late-fusion strategy, classification head, training techniques, and final model architecture.

Notebook: Deep Learning Based Depression Detection from Brain MRI
This supporting notebook section presents the practical coding implementation of the deep learning pipeline. It includes the main notebook phases, code cells, outputs, training logs, dataset preparation, model training, and independent test-set evaluation. The notebook is placed after Chapter 6 because it directly supports the model development chapter and provides practical evidence of the implemented deep learning workflow.

Chapter 7: System Testing
This chapter presents the testing process of the system. It includes functional testing, backend API testing, frontend testing, integration testing, model performance testing, explainability testing, and non-functional testing. It also reports the independent test-set results, including accuracy, AUC, sensitivity, specificity, and confusion matrix.

Chapter 8: Future Work and Conclusion
This chapter summarizes the main achievements of the project and discusses future improvements, such as external dataset validation, local Saudi dataset collection, improved Grad-CAM explainability, false negative reduction, secure backend deployment, privacy protection, federated learning, and clinical expert evaluation. It concludes the thesis by highlighting the contribution of the proposed AI-assisted framework.

CHAPTER 2: REVIEW OF LITERATURE

2.1 Introduction

This chapter presents a broad and comprehensive overview of the scientific literature that informs this project. The review is organized to build a persuasive argument for the proposed solution.

It begins by establishing the neurobiological basis of Major Depressive Disorder (MDD) on structural MRI (sMRI) scans. It provides an overview of the evolution of analytical approaches, from retrograde machine learning to modern-day deep learning. Finally, it discusses the current state-of-the-art, identifying the main research gaps that this project addresses, including 3D volumetric analysis, multimodal learning, clinical interpretability, and model generalization.

2.2 Neurobiological Biomarkers in sMRI

The core premise of using sMRI for depression detection is that MDD is associated with measurable physical changes in brain structure (i.e., morphology). A vast body of neuroscience research supports this. T1-weighted sMRI scans provide a detailed 3D map of the brain's anatomy, allowing researchers to quantify the volume, shape, and thickness of gray matter.

Key findings consistently associate MDD with structural abnormalities in brain regions responsible for emotion regulation, memory, and executive control. Studies, such as the 2021 paper by Mousavian et al., have successfully used sMRI data to identify these changes, particularly in the limbic system and prefrontal cortex [2]. The most commonly cited biomarkers include:

- **Limbic System:** The hippocampus and amygdala are most frequently cited. Studies repeatedly show a reduction in hippocampal volume (atrophy) in patients with MDD.
- **Prefrontal Cortex (PFC):** Regions such as the orbitofrontal cortex and dorsolateral prefrontal cortex (dlPFC) often show cortical thinning (a reduction in gray matter thickness).
- **Anterior Cingulate Cortex (ACC):** This region, which bridges the "emotional" limbic system and the "cognitive" prefrontal cortex, also shows significant volumetric and morphological changes [2].

These physical biomarkers are the "signal" that a deep learning model can be trained to detect.

Although structural MRI provides important anatomical information, depression diagnosis is clinically complex and may benefit from combining imaging features with patient-level clinical variables. Therefore, recent AI research increasingly

explores multimodal learning approaches that integrate brain imaging with clinical metadata to improve diagnostic performance and clinical relevance.

2.3 Evolution of sMRI Analysis for MDD

The methods for analyzing these sMRI biomarkers have evolved significantly:

1. **Manual & Statistical Methods (VBM):** Early studies relied on Voxel-Based Morphometry (VBM), a statistical method to compare brain volumes voxel-by-voxel between groups. This approach is effective for *group-level* analysis but is not designed for *individual patient* diagnosis.
2. **"Shallow" Machine Learning (SVM, Random Forest):** The next step involved using traditional machine learning models like Support Vector Machines (SVMs). However, these models cannot process raw 3D-sMRI scans. Instead, researchers first had to manually *extract features* (e.g., a table of volumes for the hippocampus, amygdala, etc.) and then feed this limited feature set to the SVM. As Mousavian et al. (2021) demonstrated, this "feature-based" approach is viable but depends heavily on *a priori* knowledge, limiting the model's ability to discover *new* or complex spatial biomarkers [2].
3. **Deep Learning (CNNs):** This is the current state-of-the-art. Deep Learning, specifically Convolutional Neural Networks (CNNs), can analyze the raw image data directly. This "end-to-end" approach allows the model to learn its own features, identifying complex patterns of atrophy or thinning across the entire 3D brain volume that a human-defined feature set would miss.

   This evolution also supports the development of more advanced systems that combine automatic 3D feature extraction with clinical information, allowing models to capture both anatomical and patient-level indicators of depression.

2.4 Deep Learning for sMRI-Based Depression Detection

Within the domain of deep learning, Convolutional Neural Networks (CNNs) are widely used for medical image analysis because they can automatically learn spatial features from imaging data.

In MRI-based depression detection, 3D-CNN models are especially important because structural MRI scans are volumetric data. Instead of treating the brain scan as separate 2D slices, a 3D-CNN analyzes the MRI scan as a full 3D block. This allows the model to learn spatial relationships between brain regions and capture subtle morphological patterns that may be distributed across the whole brain.

Recent studies have shown that 3D volumetric analysis is a promising direction for detecting depression-related brain changes from sMRI data. By learning directly from the full MRI volume, 3D-CNN models can identify complex anatomical patterns that may not be visible through manual inspection or hand-crafted feature extraction.

This supports the direction of the proposed system, which uses a 3D CNN-based imaging branch as part of a multimodal framework. The imaging branch extracts anatomical features from structural MRI volumes, while the clinical branch processes patient-level clinical metadata. These two feature representations are then combined through late fusion for MDD classification.

2.5 Advanced Frontiers: Multi-Modal and Federated Learning

The field is also expanding in other directions, primarily by adding more data types. A 2025 survey by Li et al. highlights the trend of using multi-modal data—such as brain imaging, text, and audiovisual data—to improve predictive accuracy [6].

- **Multi-Modal Detection:** Research is exploring the fusion of MRI data with other biomarkers. For example, a 2024 paper by Vandana et al. proposed a framework that combines MRI images with speech signals to detect depression [5]. While powerful, this multi-modal approach adds significant complexity and relies on acquiring two very different types of data.
- **Federated Learning:** The same 2024 study (Vandana et al.) also highlights Federated Learning, an advanced AI training method [5]. This technique allows a model to be trained on data from multiple hospitals *without* the data ever leaving the hospital, thus preserving patient privacy.

These advanced frontiers directly support the direction of this project. Instead of relying only on MRI images, the proposed system adopts a multimodal approach by combining 3D structural MRI features with clinical metadata. This allows the model to learn both anatomical patterns and patient-level clinical information, which may improve classification performance and clinical relevance. Federated learning remains outside the current project scope but represents an important future direction for privacy-preserving clinical deployment.

2.6 The "Black Box" Problem: The Need for Explainable AI (XAI)

A major barrier to the clinical adoption of deep learning models is the "black box" problem. A model may achieve 95% accuracy, but if clinicians cannot understand *how* it arrived at a diagnosis, it cannot be trusted. The model could be learning an irrelevant artifact (e.g., differences in scanner manufacturers) rather than true disease pathology.

As a comprehensive 2023 survey by Squires et al. on "Deep learning...in psychiatry" notes, interpretability and clinical trust remain major gaps [4].

This is why Explainable AI (XAI) is critical. XAI methods, such as Grad-CAM (Gradient-weighted Class Activation Mapping), are used to "look inside" the model.

Grad-CAM produces a visual heatmap that highlights which parts of the input image (in this case, which anatomical brain regions) the model "looked at" most when making its decision. This directly addresses the black box problem. If the Grad-CAM heatmaps show the model is focusing on the hippocampus and ACC (known biomarkers), it builds clinical trust. If it focuses on the corner of the image or the skull, it reveals the model is flawed.

In this project, explainability is important because multimodal deep learning models can be difficult to interpret. Grad-CAM can be used to explain the imaging branch by highlighting anatomical regions that contribute most strongly to the model's decision, while clinical feature analysis can help interpret the contribution of patient-level variables. This improves transparency and supports clinical trust in the proposed AI-assisted framework.

2.7 Context: AI in Functional Neuroimaging (fMRI)

While this project's scope is strictly limited to sMRI, it is important to note the parallel success seen in functional MRI (fMRI) analysis. This body of research validates the general approach of using AI for depression diagnostics.

- A 2023 systematic review and meta-analysis by Chen et al. analyzed 31 studies on fMRI and machine learning [7]. It found that ML models could diagnose MDD with a high pooled accuracy (AUC of 0.86) and high sensitivity (80%) and specificity (83%).
- Similarly, a 2025 study by Mansoor & Ansari used a Deep Neural Network (DNN) on fMRI data from multiple sites to achieve 89% accuracy in detecting early-stage MDD [8].
- Crucially, the Mansoor & Ansari study also implemented an XAI technique (SHAP) to find *which* brain connections were most important, reinforcing the trend that interpretability is now a key component of high-impact research in this field [8].

Although this project focuses on structural MRI rather than functional MRI, these studies support the broader conclusion that AI can identify meaningful neuroimaging patterns related to depression. They also reinforce the importance of interpretability methods in clinical AI systems.

2.8 Literature Survey

The following table summarizes the key literature reviewed in this chapter, highlighting the progression of research and the limitations that this project will address.

Author(s) & Year	Data Modality	Method Used	Key Finding / Limitation
Mousavian et al. (2021) [2]	sMRI & rs-fMRI	"Shallow" ML (SVM, RF)	Achieved good accuracy, but relied on pre-extracted, "hand-crafted" features.
Vandana et al. (2023) [5]	Audio & Text	Hybrid DL (CNN + Bi-LSTM)	High accuracy on non-imaging data. Shows multi-modal potential, but doesn't use neurobiological data.
Mansoor & Ansari (2025) [8]	fMRI (functional)	"Shallow" ML (SVM, RF)	High accuracy for functional connectivity. Does not analyze brain *structure* (sMRI).
Li et al. (2024) [3]	sMRI	3D-CNN (3D-ResNet)	State-of-the-art accuracy by analyzing the full 3D volume of the sMRI scan.
Squires et al. (2023) [4]	Survey Paper	(N/A - Review)	Identified the key research gap: Most models are "black boxes," which is the primary barrier to clinical adoption.
Proposed Project	3D sMRI + clinical metadata	Autoencoder pretraining + 3D CNN + SE attention + late fusion	Addresses gaps by combining volumetric MRI analysis, clinical metadata, and explainability support for MDD classification.

Table 2.1: Literature Survey

2.9 Critical Analysis and Research Gaps

This literature review identifies four critical gaps in the current research, which this project aims to address:

1. Limited Use of Multimodal Learning: Many MRI-based depression detection studies rely mainly on imaging data alone. However, depression is a clinically

complex disorder, and clinical variables may provide additional diagnostic context. This creates a need for models that combine MRI-based anatomical features with patient-level clinical metadata.

2. Over-reliance on Feature-Based or Slice-Based Analysis: Some studies rely on shallow machine learning models using pre-extracted hand-crafted features, which limits the model to known biomarkers and reduces its ability to discover complex spatial patterns. Other approaches analyze MRI scans slice by slice, which may lose important 3D spatial relationships between brain structures.

3. Lack of Interpretability: Many deep learning models focus primarily on diagnostic accuracy without explaining how predictions are made. This black-box nature limits clinical trust and makes it difficult for psychiatrists and radiologists to understand whether the model is focusing on meaningful anatomical regions.

4. Data Limitations and Generalization: Many studies use small, private, or single-site datasets, making their models prone to overfitting and difficult to reproduce. This highlights the need for careful preprocessing, validation, and evaluation strategies to improve model reliability and generalization.

2.10 Chapter Summary

The reviewed literature confirms that structural MRI scans contain subtle but meaningful morphological biomarkers associated with Major Depressive Disorder, especially in regions such as the hippocampus, amygdala, prefrontal cortex, and anterior cingulate cortex. The literature also shows a clear evolution from manual statistical analysis and shallow machine learning toward deep learning models capable of automatically learning complex spatial features from brain MRI data.

Recent 3D-CNN-based studies demonstrate the value of analyzing the full 3D brain volume rather than relying only on hand-crafted features or isolated slice-level analysis. At the same time, recent multimodal research highlights the potential benefit of combining imaging data with clinical variables to improve diagnostic performance and clinical relevance.

However, several research gaps remain, including limited multimodal integration, reliance on feature-based or slice-based analysis, lack of interpretability, and data generalization challenges. Therefore, this project builds upon the 3D-sMRI deep learning direction and extends it through a SOTA-inspired multimodal framework that combines 3D MRI features, clinical metadata, autoencoder-based healthy brain pretraining, Squeeze-and-Excitation attention, and explainability support.

The next chapter, Methodology and Analysis, will describe the dataset preparation, MRI preprocessing, clinical metadata integration, model architecture, training strategy, and evaluation metrics used in the proposed system.

CHAPTER 3: METHODOLOGY AND ANALYSIS

3.1 Introduction

This chapter describes the methodology used to develop and evaluate the proposed AI-assisted framework for Major Depressive Disorder detection from brain MRI data. Following the problem statement in Chapter 1 and the literature review in Chapter 2, this chapter explains the complete workflow, including dataset preparation, MRI preprocessing, clinical metadata integration, model development, training, validation, testing, and explainability support.

The methodology is designed to support the project's current direction: a SOTA-inspired multimodal deep learning framework that combines 3D structural MRI features with clinical metadata. The system uses a 3D MRI imaging branch to extract anatomical features, a clinical metadata branch to process patient-level variables, and a late-fusion classification head to distinguish between MDD and Healthy Control subjects.

This chapter also explains the use of autoencoder-based healthy brain pretraining, Squeeze-and-Excitation attention, advanced training strategies, and independent test-set evaluation.

3.2 Conceptual Framework

The project's analysis is built on a high-level conceptual framework that defines the flow of data. This framework, illustrated in Figure 3.1, consists of seven key stages.

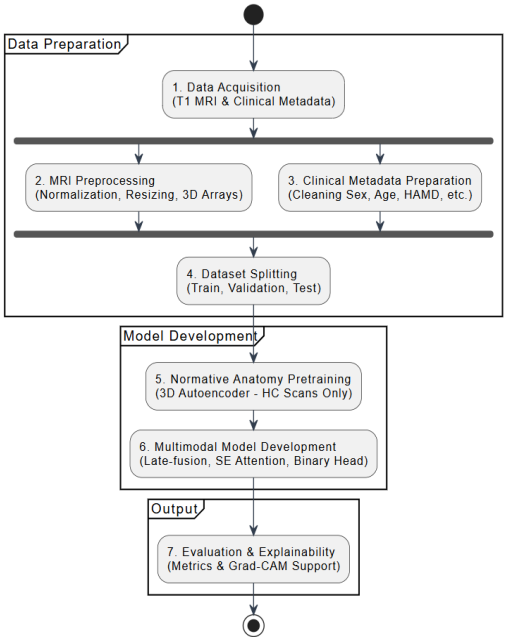


Figure 3.1: Conceptual Framework for Analysis

The proposed methodology consists of seven main stages:

1. **Data Acquisition:** Collecting T1-weighted structural MRI scans and corresponding clinical metadata for MDD and Healthy Control subjects.
2. **MRI Preprocessing:** Preparing MRI volumes through cleaning, normalization, resizing, and conversion into standardized 3D arrays suitable for deep learning.
3. **Clinical Metadata Preparation:** Extracting and cleaning relevant clinical variables such as sex, age, education, and HAMD score, then merging them with the MRI data using subject identifiers.
4. **Dataset Splitting:** Dividing the dataset into training, validation, and independent test sets while preserving class distribution.
5. **Normative Anatomy Pretraining:** Training a 3D convolutional autoencoder using only Healthy Control MRI scans to learn a representation of normal brain anatomy.
6. **Multimodal Model Development and Training:** Building a SOTA-inspired late-fusion model that combines a 3D MRI branch, Squeeze-and-Excitation attention, a clinical metadata branch, and a binary classification head.
7. **Evaluation and Explainability Support:** Evaluating the model using diagnostic metrics and preparing the system for explainability methods such as Grad-CAM.

3.3 Data Acquisition and Selection

The foundation of this project is based on large-scale, open-access neuroimaging datasets related to Major Depressive Disorder (MDD). Instead of relying on a small single dataset, this project uses datasets provided by The Depression Imaging REsearch ConsorTium (DIRECT) and the R-fMRI Lab to improve reliability, reproducibility, and generalization.

The selected datasets are:

1. **DIRECT Phase II Dataset [10]**
   This dataset is provided by The DIRECT Consortium and includes multi-site neuroimaging data for MDD patients and healthy controls. DIRECT Phase II contains both T1-weighted structural MRI and resting-state fMRI data from a large number of participants, making it suitable for deep learning-based depression detection. The dataset includes approximately 1,660 MDD patients and 1,341 healthy controls across multiple cohorts [10] .

2. **REST-meta-MDD Dataset [9]**
   This dataset is provided by the R-fMRI Lab as part of the REST-meta-MDD project. It is one of the largest public neuroimaging datasets for MDD research, containing data from 1,300 MDD patients and 1,128 healthy controls collected across 25 cohorts [9].

For this project, the focus is on using T1-weighted structural MRI scans as the main imaging input for the deep learning model. In addition, clinical metadata such as age,

sex, education, and HAMD score can be integrated to support a multimodal classification approach, as implemented in the project notebook.

The data acquisition sources are:

- REST-meta-MDD official website: https://rfmri.org/REST-meta-MDD
- DIRECT Phase II dataset: https://doi.org/10.57760/sciencedb.psych.00689

The use of these datasets strengthens the project by providing a larger and more clinically relevant sample compared with small single-site datasets. This supports the development of a more robust AI-based system for distinguishing between MDD patients and Healthy Controls (HC) using brain MRI data.

3.4 Data Pre-processing Pipeline

Raw MRI data cannot be used directly in a deep learning model because scans may differ in size, orientation, intensity distribution, and background noise. Therefore, a preprocessing pipeline is required to standardize the input data before training.

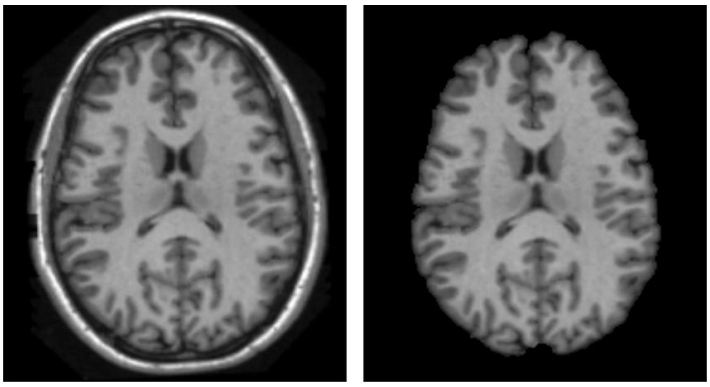


Figure 3.2: Example of Skull Stripping (Left: Raw Scan, Right: Brain-Extracted Scan)

The MRI preprocessing pipeline includes the following steps:

1. **Brain Extraction and Cropping:** Non-brain background regions are removed or reduced to focus the model on brain tissue and minimize irrelevant information.
2. **Resizing:** Each MRI volume is resized to a fixed 3D shape to ensure that all samples have the same input dimensions for the neural network.
3. **Intensity Normalization:** Voxel intensities are normalized using Z-score normalization so that the model learns structural patterns rather than scanner-specific brightness differences.
4. **Conversion to Deep Learning Format:** The processed MRI volumes are stored as numerical arrays and loaded as 3D tensors during training.
5. **Clinical Metadata Processing:** Clinical variables such as sex, age, education, and HAMD score are cleaned, missing values are handled, and

continuous variables are normalized.

6. **MRI-Clinical Data Fusion:**The MRI file paths, diagnostic labels, and clinical metadata are merged using subject identifiers to create the final multimodal dataset.

3.5 Model Development

To answer our first research question (2D vs. 3D), two distinct model architectures are defined for comparison.

3.5.1 Normative Anatomy Pretraining using 3D Autoencoder

Before training the final classifier, a 3D convolutional autoencoder is trained using only Healthy Control MRI scans. The purpose of this step is to learn a normative representation of healthy brain anatomy.

The autoencoder consists of:

- Encoder: Compresses the input MRI volume into latent anatomical features.

- Decoder: Reconstructs the MRI volume from the compressed representation.

The encoder weights learned during this phase are saved and later used as an anatomical blueprint for the final diagnostic model.

3.5.2 MRI Imaging Branch

The MRI branch uses 3D convolutional layers to extract spatial anatomical features from full brain volumes [3]. Unlike 2D methods that analyze slices independently, 3D convolutions preserve spatial relationships across depth, height, and width.

3.5.3 Squeeze-and-Excitation Attention

A Squeeze-and-Excitation attention block is applied to the MRI feature maps [12]. This mechanism helps the model emphasize more informative feature channels and reduce the influence of less useful features.

3.5.4 Clinical Metadata Branch

The clinical branch uses a Multi-Layer Perceptron to process patient-level variables such as sex, age, education, and HAMD score. This branch allows the model to learn useful non-imaging patterns related to depression.

3.5.5 Late-Fusion Classification Head

The features extracted from the MRI branch and the clinical metadata branch are concatenated into a single fused representation. This fused vector is passed into a binary classification head that predicts whether the subject belongs to the MDD or Healthy Control class.

3.6 Model Training and Validation

The training process is divided into two main stages: unsupervised pretraining and supervised multimodal classification.

1. **Dataset Split :** The dataset is divided into training, validation, and independent test sets. The training set is used to optimize model parameters, the validation set is used to monitor performance during training, and the test set is kept separate until final evaluation.
2. **Autoencoder Pretraining :** Only Healthy Control samples from the training set are used to train the 3D autoencoder. The autoencoder is trained to reconstruct healthy brain MRI volumes using Mean Squared Error loss. This allows the encoder to learn a representation of normal brain anatomy.
3. **Supervised Multimodal Training :** The final multimodal model is trained using both MRI volumes and clinical metadata. The model predicts a binary label representing either MDD or Healthy Control.
4. **Optimization :** The model is trained using the AdamW optimizer. Advanced training strategies such as data augmentation, focal loss, label smoothing, MixUp [14], gradient clipping, and cosine annealing learning-rate scheduling are used to improve robustness and reduce overfitting [13].
5. **Validation :** After each training epoch, the model is evaluated on the validation set. The best model checkpoint is saved based on validation performance.

3.7 Evaluation Metrics

To evaluate the models, simple "accuracy" is not sufficient for a medical diagnostic tool. A **Confusion Matrix** (Figure 3.4) will be used to derive a more complete set of metrics.

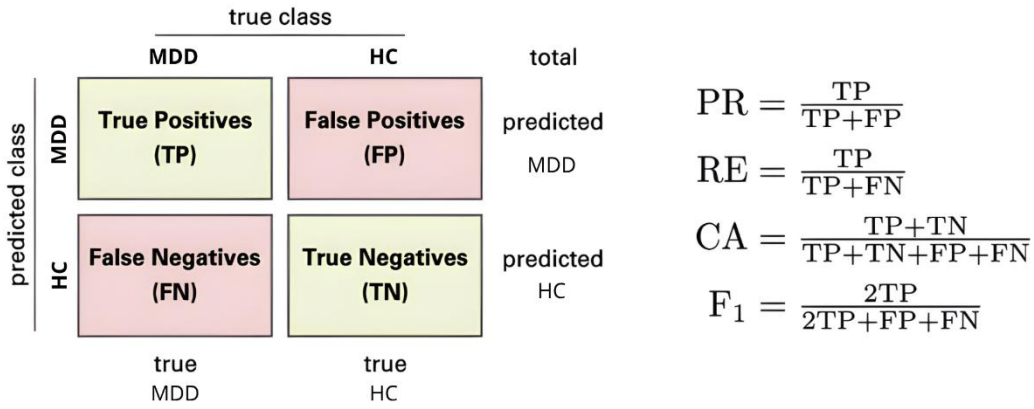


Figure 3.4: A 2x2 Confusion Matrix

- **True Positive (TP):** Model predicts "MDD," and the patient *is* MDD. (Correct)
- **True Negative (TN):** Model predicts "Healthy," and the patient *is* Healthy.

(Correct)
- **False Positive (FP):** Model predicts "MDD," but the patient is Healthy. (Type I Error)
- **False Negative (FN):** Model predicts "Healthy," but the patient *is* MDD. (Type II Error - very dangerous)

From this, we will calculate:

- **Accuracy:** (TP + TN) / Total — The percentage of all correct predictions.
- **Sensitivity (Recall):** TP / (TP + FN) — **This is a critical metric.** It answers: "Of all the *actual* MDD patients, what percentage did our model correctly find?"
- **Specificity:** TN / (TN + FP) — "Of all the *healthy* controls, what percentage did our model correctly identify?"
- **F1-Score:** The balanced (harmonic) mean of Precision and Recall, providing a single score for overall model robustness.

ROC-AUC: The Receiver Operating Characteristic - Area Under the Curve measures the model's ability to distinguish between MDD and Healthy Control subjects across different classification thresholds. A higher AUC value indicates better diagnostic discrimination performance.

3.8 Explainable AI (XAI) Framework

Explainability is important because deep learning models can operate as black boxes. In this project, explainability support is considered for the MRI imaging branch of the multimodal model.

Grad-CAM can be applied to the trained 3D imaging branch to identify which MRI regions contribute most strongly to the final classification decision [15]. The resulting heatmaps can be overlaid on MRI slices to visually highlight important anatomical regions.

This can help clinicians and researchers verify whether the model focuses on meaningful brain areas, such as the hippocampus, prefrontal cortex, or anterior cingulate cortex, rather than irrelevant image artifacts. For the clinical metadata branch, feature contribution analysis may also be considered in future work to better understand the role of patient-level variables.

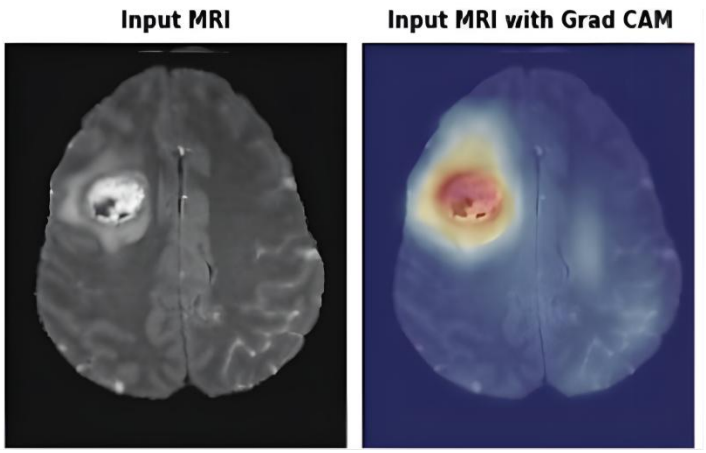


Figure 3.5: Example of a Grad-CAM Heatmap on an sMRI Scan

3.9 Work Plan For Next Stage

Phase	Duration	Key Tasks
Data Engineering	Weeks 1–2	Prepare MRI paths, labels, and clinical metadata; create train, validation, and test CSV files.
Dataset Engine	Weeks 2–3	Build the multimodal dataset loader for MRI volumes and clinical variables.
Autoencoder Pretraining	Weeks 3–4	Train the 3D autoencoder on Healthy Control MRI scans and save encoder weights.
Multimodal Model Development	Weeks 4–6	Implement the MRI branch, SE attention, clinical branch, and late-fusion classifier.
Training and Validation	Weeks 6–8	Train the multimodal model using advanced optimization and validation monitoring.
Independent Testing	Week 9	Evaluate the best model using confusion matrix, classification report, sensitivity, specificity, and ROC-AUC.
Explainability and Interface	Week 10	Prepare explainability support and connect outputs to the prototype interface.

Table 3.1: Work Plan For Next Stage

3.10 Chapter Summary

This chapter described the methodology used to develop the proposed multimodal AI-assisted framework for MDD detection from brain MRI data. It explained the dataset preparation process, MRI preprocessing, clinical metadata integration, and multimodal dataset construction.

The chapter also described the model development process, including 3D autoencoder-based normative anatomy pretraining, the MRI imaging branch, Squeeze-and-Excitation attention, the clinical metadata branch, and late-fusion classification. In addition, it presented the training strategy, validation process, evaluation metrics, and explainability support.

The following chapter, Chapter 4: System Design and Implementation, will describe the implementation of the proposed system, including the dataset pipeline, model architecture, training workflow, evaluation outputs, and prototype interface.

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

This chapter presents the complete system design of the proposed AI-assisted depression detection framework using structural MRI and clinical metadata. While Chapter 3 described the methodology for data preparation, model training, validation, and evaluation, this chapter focuses on the system architecture, requirements, user interactions, functional flow, and interface design.

The goal of this design is to create a clinically oriented AI-assisted prototype that allows users to:

1. Upload or select a preprocessed T1-weighted MRI scan.
2. Provide or load the corresponding clinical metadata.
3. Run the trained multimodal deep learning model.
4. Display the classification result as Healthy Control or Major Depressive Disorder.
5. Show the confidence score and diagnostic outputs in an interactive interface.
6. Support explainability visualization such as Grad-CAM heatmaps for the MRI imaging branch.

4.2 System Requirements

System requirements define what the system must do (functional) and how well it must do it (non-functional).

4.2.1 Hardware Requirements

The system requires sufficient computational resources for training and running 3D medical imaging models. Since the proposed model processes 3D MRI volumes and clinical metadata, GPU acceleration is recommended for model training and efficient inference.

Minimum Requirements:

- CPU: Intel i5 or higher
- RAM: ≥ 16 GB
- GPU: NVIDIA GPU with at least 4 GB VRAM
- Storage: ≥ 20 GB free space

Preferred Requirements:

- GPU: NVIDIA RTX 2080 / RTX 3090 / A100
- RAM: ≥ 32 GB
- High-speed SSD storage
- CUDA-supported environment

Cloud Options:

- Google Colab Pro
- KKU GPU Lab servers
- RunPod or similar GPU platforms

4.2.2 Software Requirements

Model Development:

- Python 3.10+
- PyTorch
- NumPy and Pandas
- Scikit-learn
- Matplotlib
- TorchVision
- Google Colab or Jupyter Notebook

Medical Imaging and Data Processing:

- NIfTI / MRI preprocessing tools when needed
- NumPy array (.npy) support for preprocessed MRI volumes
- CSV files for labels and clinical metadata

Prototype Interface:

- Next.js
- React
- TypeScript
- Tailwind CSS
- Vercel deployment

Version Control:

- GitHub

4.3 Functional Requirements

ID	Requirement	Description
FR1	MRI Data Input	The system must allow users to upload or select a preprocessed T1-weighted 3D MRI volume.
FR2	Clinical Metadata Input	The system must support patient-level clinical variables such as sex, age, education, and HAMD score.

FR3	Multimodal AI Analysis	The system must process both MRI data and clinical metadata using the trained multimodal deep learning model.
FR4	Results Display	The system must display the predicted class, either Healthy Control or Major Depressive Disorder, along with a confidence score.
FR5	MRI Visualization	The system must display MRI preview slices to help users inspect the uploaded scan.
FR6	Explainability Visualization	The system should support Grad-CAM-style heatmap visualization for the MRI imaging branch.
FR7	New Scan	The system must allow the user to reset the workflow and start a new analysis.

Table 4.1: Functional Requirements

4.4 Non-Functional Requirements

ID	Requirement	Description
NFR1	Usability	The interface must be simple, clear, and understandable for clinicians, researchers, and academic evaluators.
NFR2	Performance	The system should provide efficient inference when deployed with suitable GPU resources.
NFR3	Reliability	The system should consistently display prediction outputs, confidence scores, and MRI previews without workflow interruption.
NFR4	Security & Privacy	Uploaded MRI data and clinical information must be handled securely and should not expose patient identifiers.

NFR5	Maintainability	The system should be modular, allowing future updates to the model, interface, and explainability components.
NFR6	Ethical Use	The interface must clearly state that the result is for research and decision-support purposes only, not a final clinical diagnosis.

Table 4.2: Non-Functional Requirements

4.5 System Models

4.5.1 Use Case Diagram

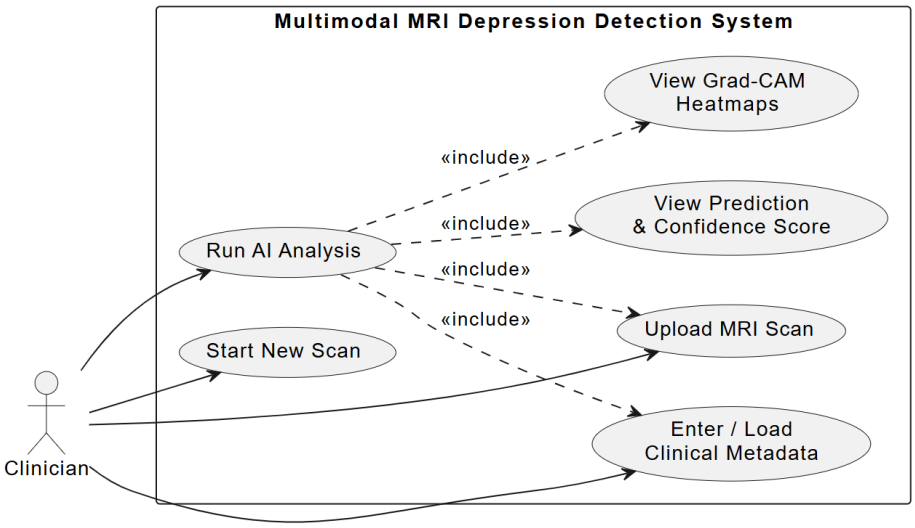


Figure 4.1: Use Case Diagram of the MRI Depression Detection System

4.5.2 Use Case Descriptions

Use Case: Upload MRI Scan

- Actor: Clinician / Researcher
- Precondition: User interface is open.
- Flow:
  - User selects or uploads an MRI file.
  - System validates the uploaded file format.
  - System prepares the MRI scan for analysis.
- Postcondition: MRI data is ready for AI analysis.

Use Case: Enter or Load Clinical Metadata

- Actor: Clinician / Researcher
- Precondition: MRI scan is uploaded or selected.

- Flow:
    - User enters clinical variables such as sex, age, education, and HAMD score, or the system loads them from the dataset.
    - System validates the clinical inputs.
    - System normalizes the clinical variables when required.
- Postcondition: Clinical metadata is ready for multimodal inference.

Use Case: Run AI Analysis

- Actor: Clinician / Researcher
- Precondition: MRI data and clinical metadata are available.
- Flow:
    - System prepares the MRI data and clinical metadata.
    - The trained multimodal AI model performs inference.
    - The model combines MRI-based features with clinical metadata.
    - The system generates the predicted class and confidence score.
- Postcondition: Prediction result and confidence score are available for display.

Use Case: View Prediction and Confidence Score

- Actor: Clinician / Researcher
- Precondition: AI analysis has been completed.
- Flow:
    - System displays the predicted class: Major Depressive Disorder or Healthy Control.
    - System displays the confidence score.
    - System presents the result in a clear visual panel.
    - System displays a research-use disclaimer to avoid clinical misinterpretation.
- Postcondition: User can review the diagnostic output.

Use Case: View Grad-CAM Heatmaps

- Actor: Clinician / Researcher
- Precondition: AI analysis has been completed.
- Flow:
    - System generates or displays Grad-CAM-style heatmap output for the MRI imaging branch.
    - The heatmap is overlaid on MRI slices to support visual interpretation.
    - User can compare the heatmap with the original MRI preview.
    - User can show or hide the heatmap when needed.
- Postcondition: User can visually inspect the MRI regions that contributed to the prediction.

Use Case: Start New Scan

- Actor: Clinician / Researcher
- Precondition: User has completed or canceled the current analysis.
- Flow:
    - User clicks Start New Scan.
    - System clears the previous MRI input, clinical metadata, prediction result, confidence score, and heatmap.

    - System returns to the initial upload screen.
- Postcondition: The system is ready for a new MRI analysis.

4.5.3 Activity Diagram

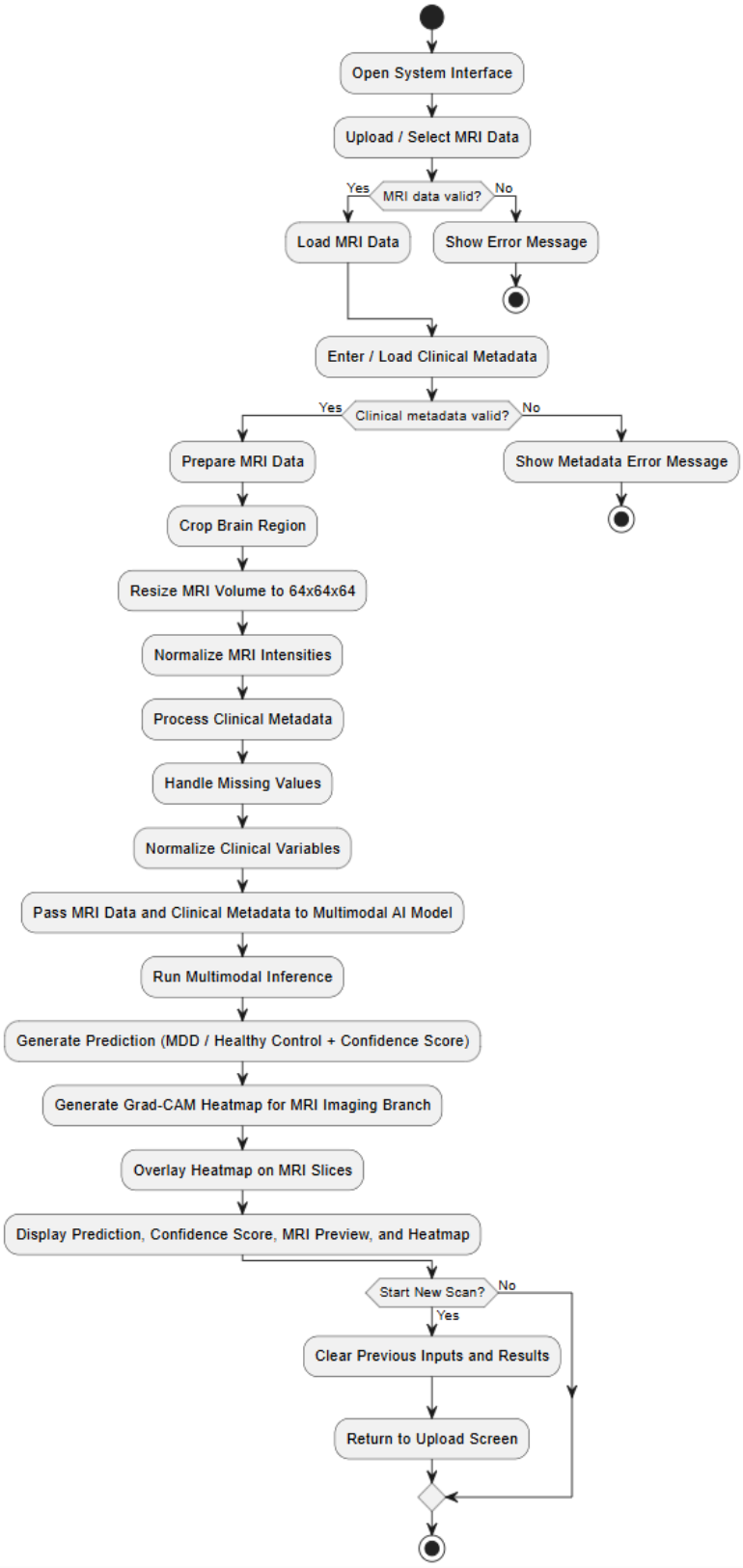


Figure 4.2: Activity Diagram of the MRI Depression Detection System

4.5.4 Sequence Diagram

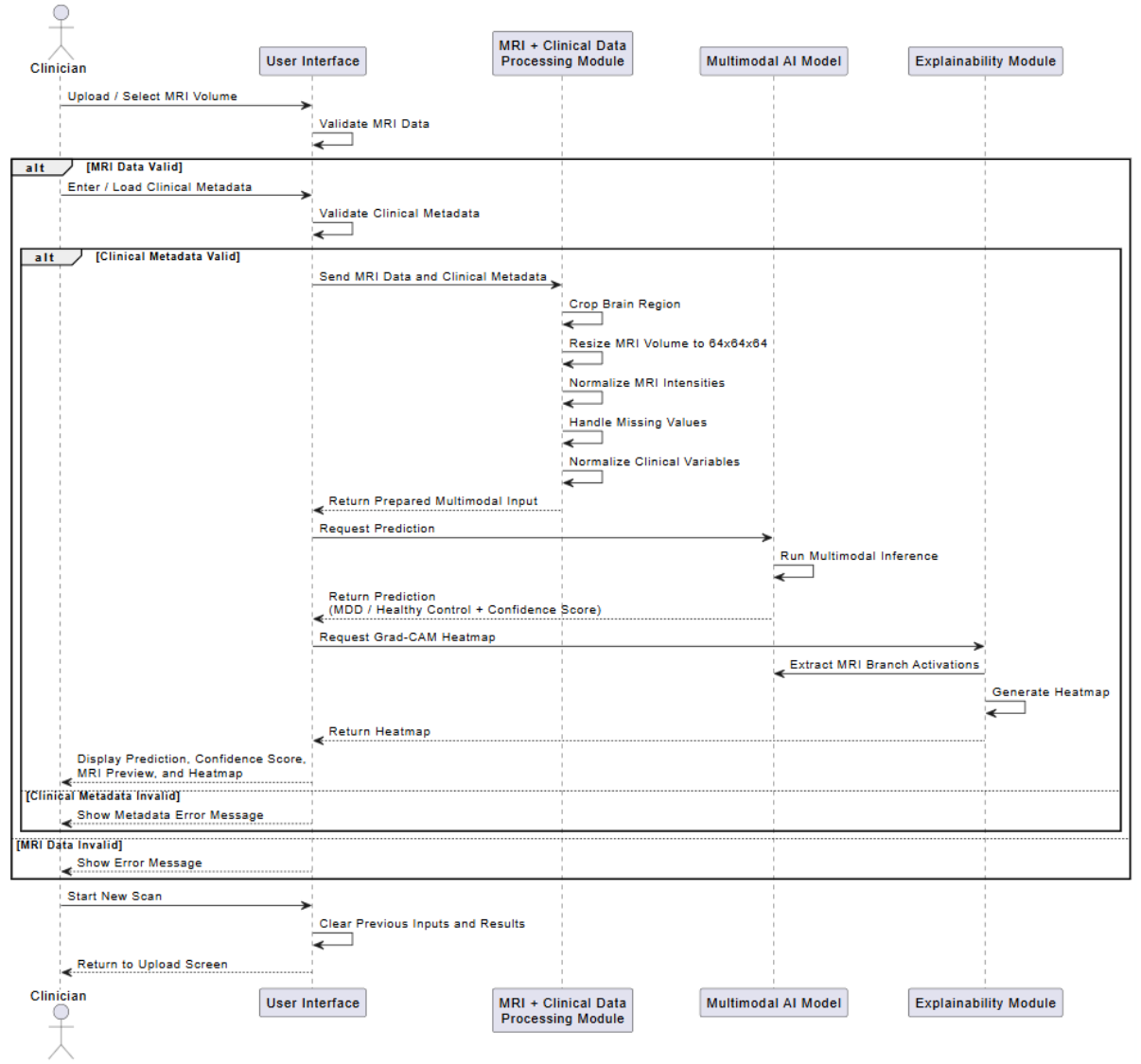


Figure 4.3: Sequence Diagram of the MRI Depression Detection System

4.5.5 Class Diagram

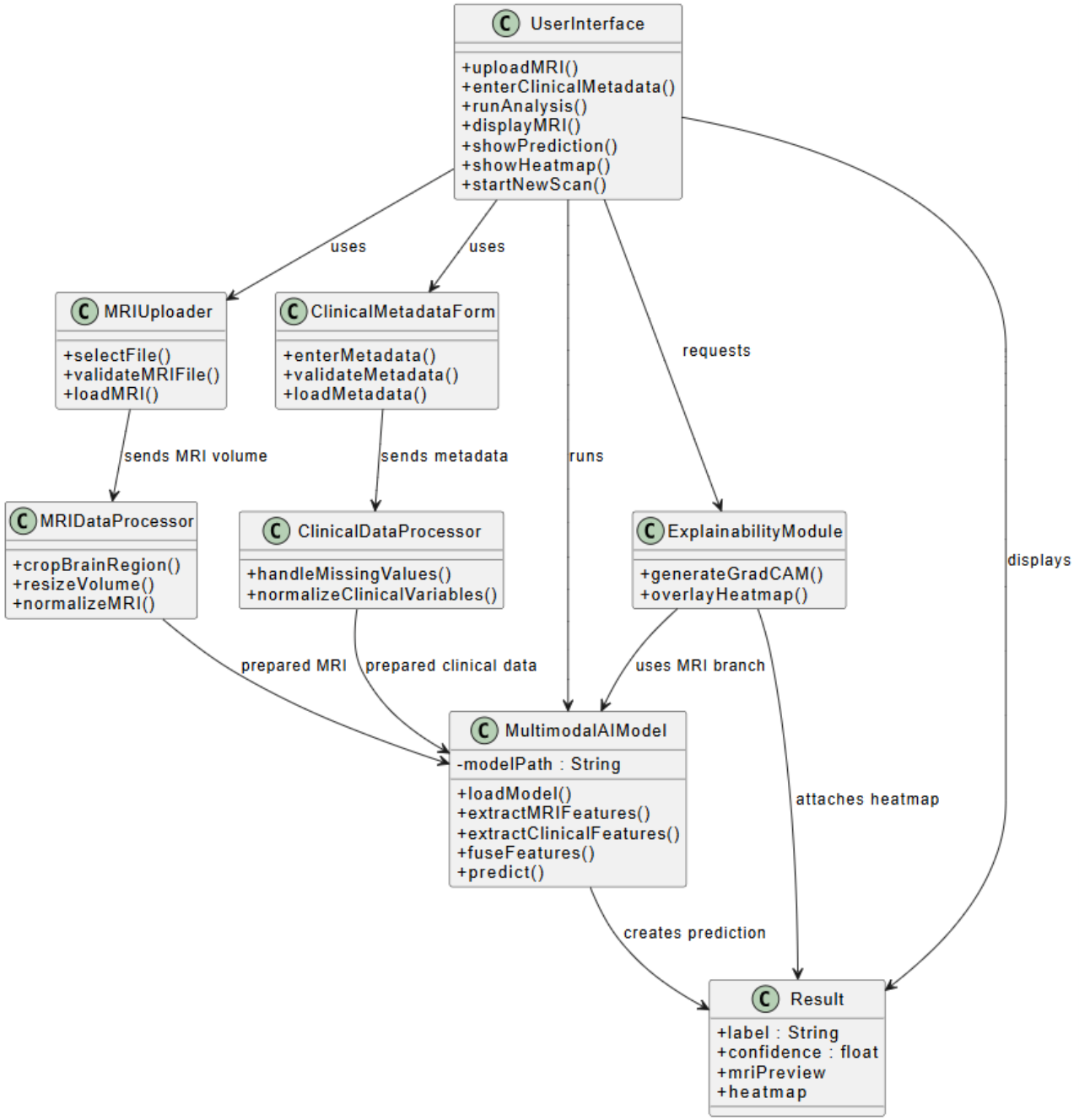


Figure 4.4: Class Diagram of the MRI Depression Detection System

4.5.6 Context Diagram

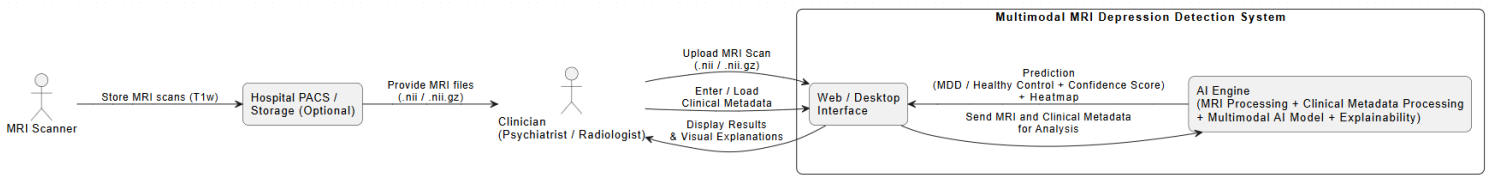


Figure 4.5: Context Diagram of the MRI Depression Detection System

4.5.7 Data Flow Diagram (DFD – Level 0, Level 1)D

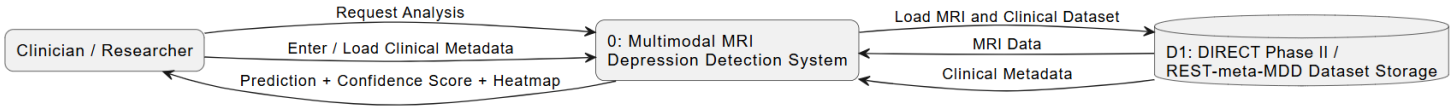


Figure 4.6: Data Flow Diagram (Level 0) of the MRI Depression Detection System

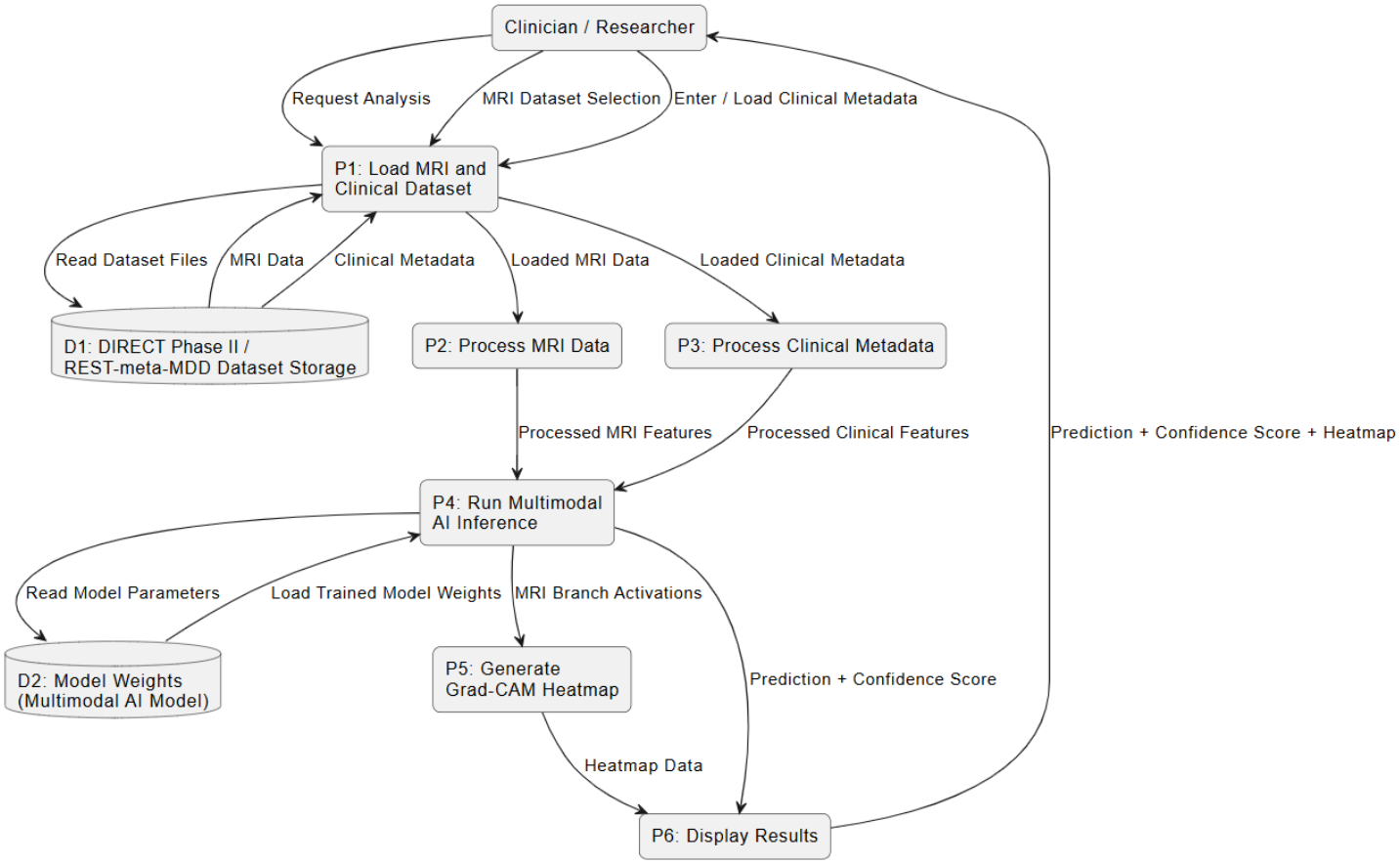


Figure 4.7: Data Flow Diagram (Level 1) of the MRI Depression Detection System

4.6 Data Organization

The project data are organized to support multimodal learning. The system combines preprocessed MRI volume paths, diagnostic labels, and clinical metadata into structured CSV files.

4.6.1 MRI Data

The MRI data consist of preprocessed T1-weighted structural MRI volumes. After preprocessing, each scan is stored as a numerical array that can be loaded directly into the deep learning model.

4.6.2 Clinical Metadata

The clinical metadata include patient-level variables such as:

- Sex
- Age
- Education
- HAMD score

These variables are cleaned, normalized, and linked to MRI samples using subject identifiers.

4.6.3 Label Files

The dataset includes CSV files that store:

- MRI volume path
- Subject ID
- Diagnostic label
- Clinical metadata

4.6.4 Final Dataset Files

The final multimodal dataset is divided into:

- Training set
- Validation set
- Independent test set
- Healthy Control subset for autoencoder pretraining

4.7 System Architecture

The final system follows a layered architecture that separates the user interface, data processing, AI model inference, and output visualization.

1. **Presentation Layer**

   This layer represents the web-based prototype interface. It includes:

   - MRI upload screen
   - Analysis progress screen
   - Results dashboard
   - MRI preview viewer
   - Prediction and confidence display
   - Heatmap visualization panel

2. **Data Processing Layer**

   This layer prepares the input data before inference. It includes:

   - MRI loading
   - MRI resizing and normalization
   - Clinical metadata loading
   - Clinical metadata normalization

- Conversion into tensors suitable for model inference

3. **AI Model Layer**

This layer contains the trained multimodal deep learning model. It includes:

- 3D MRI imaging branch
- Squeeze-and-Excitation attention block
- Clinical metadata branch
- Late-fusion classifier
- Saved model weights

4. **Explainability Layer**

This layer supports interpretability outputs such as Grad-CAM-style heatmaps for the MRI imaging branch.

5. **Data Layer**

This layer stores or references:

- Preprocessed MRI volumes
- Clinical metadata CSV files
- Diagnostic labels
- Model checkpoints
- Temporary uploaded files

4.8 Interface Design

This section describes the visual and interactive components of the MRI-based depression detection system. The interface is designed to ensure usability, clarity, and smooth workflow for both researchers and clinicians. It follows a minimal, medical-grade UI style, focusing on reducing cognitive load while presenting complex model outputs such as Grad-CAM heatmaps and prediction scores.

4.8.1 System Interface

The system interface consists of three primary screens: Upload, Analyze, and Result.

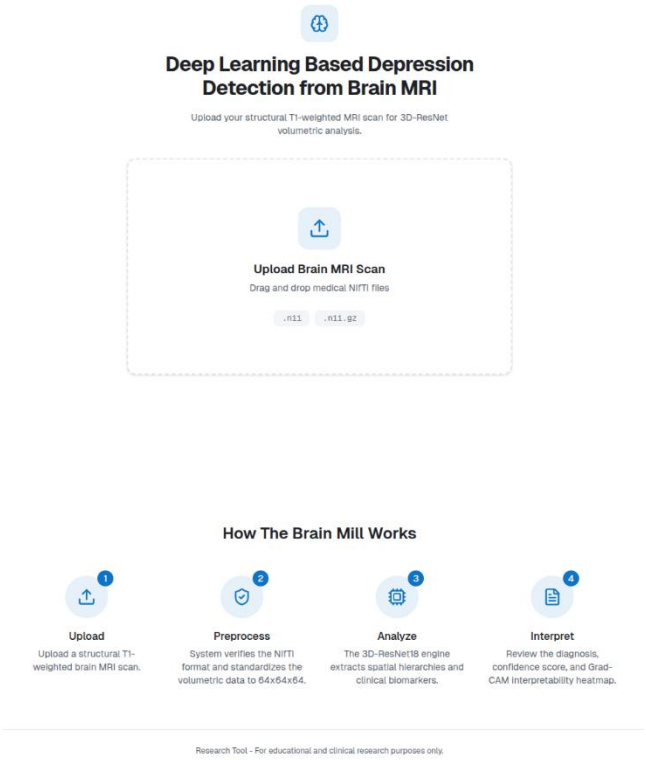


Figure 4.8: System Interface Prototype Displaying Upload, Analysis, and Result Screens

1. Upload Screen

- AAllows the user to upload a structural T1-weighted MRI scan for volumetric analysis.
- Displays a drag-and-drop file upload area supporting specific medical formats (e.g., .nii, .nii.gz).
- Includes a section explaining the system's workflow steps (Upload, Preprocess, Analyze, Interpret).
- Provides a clean, centered layout with a clear call-to-action icon.

2. Analysis Screen

- Displays the uploaded MRI file details (name and size) along with a deletion option.
- Features a "Live MRI Stream" viewer allowing 3D and multiplanar navigation (Axial, Coronal, Sagittal).
- Includes a section to input patient clinical metadata (e.g., Age, Biological Sex, Education Years, HAM-D Score).
- Shows a progress state during the "Multimodal Fusion Analysis" to prevent premature actions while the engine processes the data.
- Contains a primary button to initiate the multimodal analysis.

3. Result Screen

This is the core interface, displaying model outputs clearly and responsibly.

Elements visible (as in prototype):

- **AI Diagnostic Summary Panel:**
  - Diagnosis status badge indicating the detection result.
  - Confidence level indicator presented as a percentage gauge.
  - Architecture details summarizing the models used (e.g., Late-Fusion Multimodal, SE-Attention 3D CNN, Grad-CAM Map).
  - A prominent disclaimer stating the tool is for research purposes and requires medical verification.
- **MRI Preview Viewer**
  - "Live MRI Stream" viewer showing the brain scan.
  - Multiplanar controls to navigate different views.
  - Heatmap overlays visible within the viewer to highlight relevant brain regions.
- **Action Button:** A button to analyze a new patient, resetting the workflow.

4.8.2 Output Visualizations (Model prediction + Grad-CAM)

1. Classification Output:

- PThe system returns the diagnostic summary indicating whether depression signs were detected.
- A visual gauge clearly displays the confidence level (e.g., High Reliability) as a percentage.
- Provides a summary of the technical architecture used to reach the conclusion.
- CThis layout ensures rapid interpretability for clinical researchers.

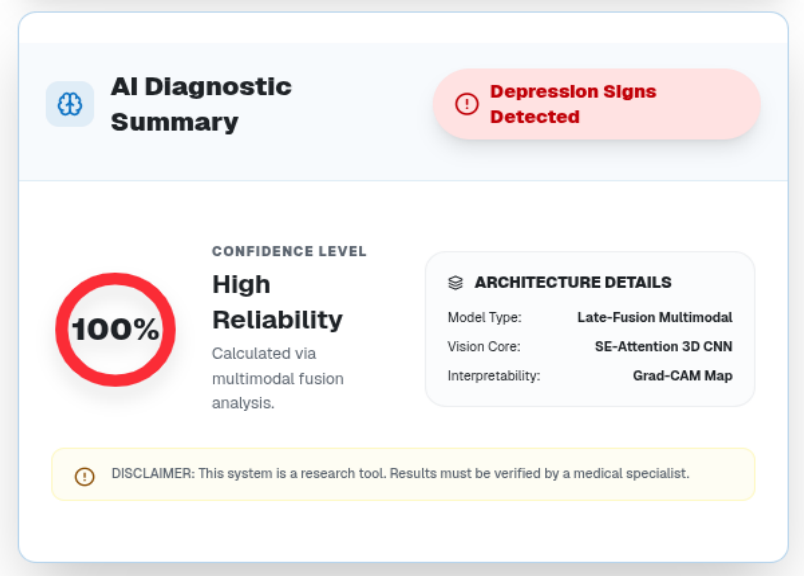


Figure 4.9: Classification Output Interface Showing MDD Prediction and Confidence Score

2. MRI Slice Visualization

- Displays the MRI volume within the "Live MRI Stream" component.
- Supports multiplanar navigation, allowing users to switch between 3D, Axial, Coronal, and Sagittal views.
- Dark UI emphasizes the brain structure and the overlaid heatmaps, reducing visual noise.
- Used for browsing the anatomy and cross-checking Grad-CAM overlays.

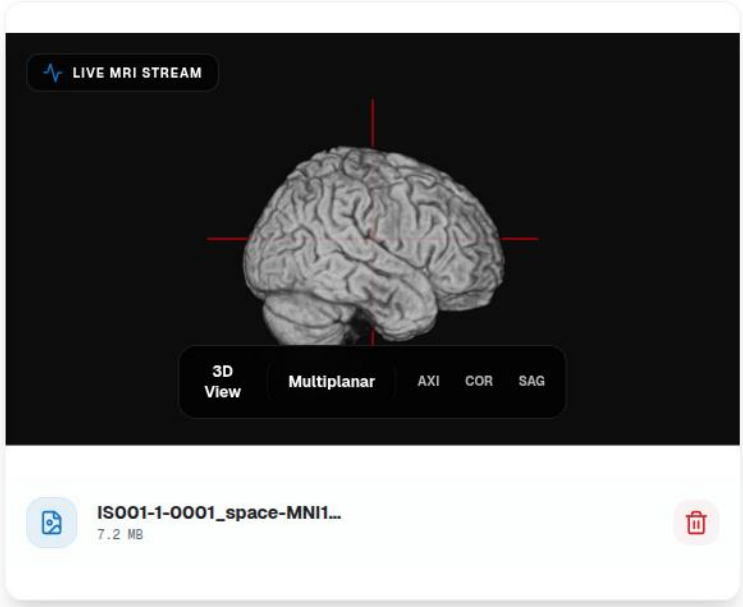


Figure 4.10: MRI Slice Visualization Interface for Navigating 2D Anatomical Views

3. Grad-CAM Heatmap

- Provides explainability for the model's decisions.
- The heatmap overlay highlights regions contributing to the prediction directly on the multiplanar views.
- Colors represent the scale of activation influence.
- Integrated directly into the main MRI viewing area for seamless comparison.

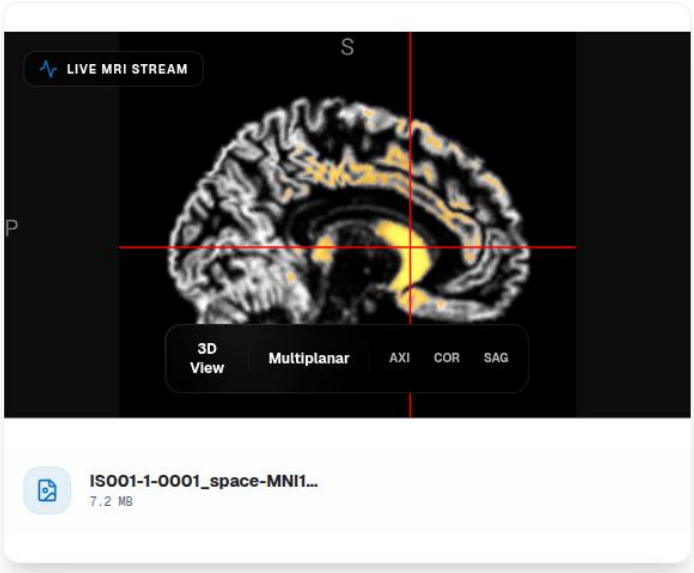


Figure 4.11: Grad-CAM Heatmap Visualization Highlighting Model Activation Regions

4. Safety & Explainability Notes

- To avoid misinterpretation, a highlighted disclaimer below the diagnostic summary explains the tool's intended research purpose.
- Ensures the ethical presentation of machine-learning predictions in a healthcare context.

4.8.3 User Interaction Flow

The user flow is designed to be linear, intuitive, and have minimal friction:

Step 1 — Upload MRI

1. UUser interacts with the upload area by dragging and dropping or selecting an MRI file (.nii or .nii.gz).
2. The system transitions to the analysis preparation screen.

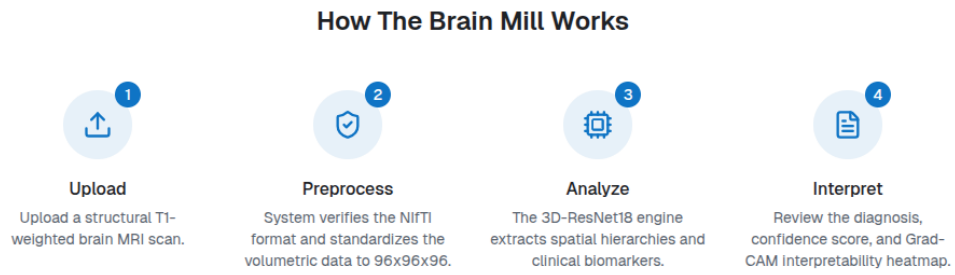


Figure 4.12: User Interaction Flow — MRI Upload Interface

Step 2 — Analyze

1. The system displays the uploaded file and the Live MRI Stream.
2. User inputs the relevant patient clinical metadata.
3. User clicks the button to run the multimodal analysis.
4. A progress state with a loading bar is shown during the fusion analysis.

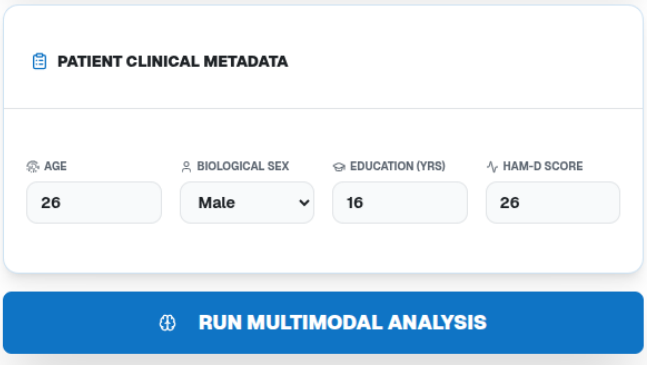


Figure 4.13: User Interaction Flow — Analysis Screen

Step 3 — View Results

1. System displays:
   i. AI Diagnostic Summary (Detection status and Confidence level).
   ii. Architecture details.
   iii. MRI preview
   iv. MRI preview with multiplanar views and heatmap overlays.

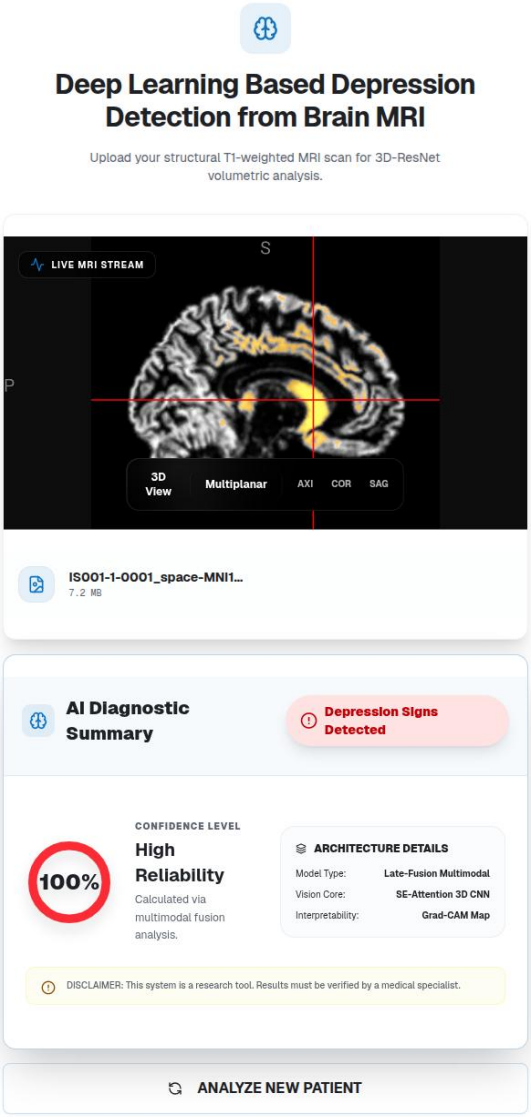


Figure 4.14: User Interface Overview Showing Combined Results Screen

1. User can:
    i. Navigate between 3D, Axial, Coronal, and Sagittal views.
    ii. Review the interpretability heatmaps.
    iii. Click to start an analysis for a new patient.

CHAPTER 5: SYSTEM IMPLEMENTATION

5.1 Introduction

This chapter describes the implementation of the proposed AI-assisted depression detection system from brain MRI. While Chapter 4 presented the system design, functional requirements, system models, and interface design, this chapter explains how the designed system was converted into a working prototype.

The implemented system consists of two main parts. The first part is the web-based user interface, developed using Next.js, React, TypeScript, and Tailwind CSS. The second part is the backend AI service, implemented using FastAPI and Python to receive MRI-related inputs, process requests, and return prediction results to the interface.

The system follows the same design direction described in the previous chapters: allowing the user to upload or select a T1-weighted brain MRI scan, provide clinical metadata, run an AI-based analysis, and display the prediction result with a confidence score and visual explanation support. This matches the functional requirements defined earlier, including MRI input, clinical metadata input, multimodal AI analysis, result display, MRI visualization, Grad-CAM-style visualization, and starting a new scan workflow.

5.2 Implementation Environment

The implementation environment was divided into frontend development, backend development, AI model development, and deployment/testing tools.

5.2.1 Frontend Environment

The frontend prototype was implemented using modern web development technologies:

Tool / Technology	Purpose
Next.js	Main frontend framework
React	Building reusable UI components
TypeScript	Type-safe frontend development
Tailwind CSS	Styling and responsive layout
GitHub	Version control and source-code management
Browser	Testing the user interface

Table 5.1 : Frontend Environment

The frontend was responsible for displaying the MRI upload interface, clinical metadata form, analysis screen, result dashboard, confidence score, MRI preview, and Grad-CAM-style heatmap visualization.

5.2.2 Backend Environment

The backend was implemented in a Python file named `main.py` using FastAPI. It loads the trained PyTorch model `best_multimodal_model.pth` and exposes the `/api/analyze` endpoint for receiving MRI scans and clinical metadata.

Tool / Technology	Purpose
Python	Backend and AI model integration
FastAPI	Creating API endpoints
Uvicorn	Running the FastAPI server
PyTorch	Loading and running the trained model
NumPy / Pandas	Data handling
ngrok	Exposing the local backend to the internet

Table 5.2 : Backend Environment

The backend receives requests from the frontend, prepares the input data, runs the AI inference process, and returns the prediction output to the frontend in JSON format.

5.3 Source Code Management

The project source code was managed using GitHub to make the implementation organized, version-controlled, and accessible to the development team.

Repository link:

`https://github.com/Ibrahim-Hurubi/mri-depression-detection`

The GitHub repository was used to store the main project files, including frontend components, backend API files, configuration files, and supporting implementation

resources. Using GitHub also helped the team track changes, collaborate on the project, and maintain a clear development history.

5.4 Frontend Implementation

The frontend represents the main interaction point between the user and the AI system. It was implemented as a web-based prototype to provide a clear and simple workflow for clinicians, researchers, and academic evaluators.

The interface follows three main screens:

1. **Upload Screen**
   This screen allows the user to upload or select a brain MRI scan. It includes a simple upload area and introduces the workflow steps: upload, preprocess, analyze, and interpret.
2. **Analysis Screen**
   After uploading the MRI scan, the user can enter clinical metadata such as age, biological sex, education years, and HAMD score. The system then allows the user to run the multimodal analysis.
3. **Result Screen**
   The result screen displays the diagnostic output, confidence score, MRI preview, and heatmap visualization. It also includes a disclaimer that the system is intended for research and decision-support purposes, not as a final clinical diagnosis.

This implementation follows the interface design described in Chapter 4, where the system interface consists of Upload, Analyze, and Result screens, including MRI preview, prediction display, confidence score, and heatmap visualization.

5.5 Backend API Implementation

The backend implementation was completed in `main.py` using FastAPI. The system defines an `/api/analyze` endpoint that receives the uploaded MRI scan and clinical metadata values including sex, age, education, and HAMD score. The backend loads the trained `best_multimodal_model.pth` model and applies the multimodal late-fusion architecture consisting of a 3D CNN MRI encoder, Squeeze-and-Excitation attention block, clinical metadata branch, and binary classifier. After inference, the backend returns the diagnosis, confidence score, and a Grad-CAM heatmap encoded as Base64 to be displayed in the frontend interface.

The following snippet shows the main FastAPI endpoint used for MRI analysis:

```
@app.post("/api/analyze")
```

```
async def analyze_mri(
    file: UploadFile = File(...),
    sex: float = Form(1.0),
    age: float = Form(0.0),
    edu: float = Form(0.0),
    hamd: float = Form(0.0)
):
    diagnosis, confidence, heatmap_data =
run_3d_ai_analysis(
        temp_input, sex=sex, age=age, edu=edu,
hamd=hamd
    )
    return {
        "diagnosis": diagnosis,
        "confidence": confidence,
        "heatmap": heatmap_base64
    }
```

The backend performs the following tasks:

1. Receives MRI scan input and clinical metadata from the frontend.
2. Validates the uploaded data and user inputs.
3. Prepares the MRI and clinical metadata for model inference.
4. Runs the trained multimodal model.
5. Returns the prediction result, confidence score, and heatmap-related output to the frontend.

A typical API response contains:

```
{
  "diagnosis": "Depression Signs Detected",
```

```
  "confidence": 87.3,
  "heatmap": "base64_encoded_heatmap"
}
```

This backend implementation supports the system architecture defined in Chapter 4, where the system is divided into presentation, data processing, AI model, explainability, and data layers.

5.6 AI Model Implementation

The AI model was implemented as a multimodal deep learning model that combines MRI-based anatomical features with clinical metadata. The model implementation follows the methodology described in Chapter 3.

The model includes:

- A 3D MRI imaging branch for extracting anatomical features from structural MRI volumes.
- A Squeeze-and-Excitation attention block to emphasize important feature channels.
- A clinical metadata branch using a Multi-Layer Perceptron to process age, sex, education, and HAMD score.
- A late-fusion classification head that combines MRI features and clinical features before producing the final binary prediction.

The system also uses a 3D autoencoder trained on Healthy Control MRI scans to learn a normative representation of healthy brain anatomy. The encoder weights are then used as an anatomical blueprint for the final classifier. This matches the methodology where the autoencoder learns healthy brain representations, and the multimodal classifier combines MRI and clinical metadata for MDD classification.

5.7 Dataset and Input Preparation Implementation

The implementation uses structured CSV files to connect MRI volumes, diagnostic labels, and clinical metadata. The clinical variables include:

- Sex
- Age
- Education
- HAMD score

The dataset was divided into training, validation, and independent test sets. According to the implemented notebook, the final split produced:

Dataset Split	Number of Samples
Training set	1732
Validation set	217
Test set	217
Healthy-only training samples	736

Table 5.3: Final Dataset Split

The implementation also created four main CSV files:

- `train_multimodal.csv`
- `val_multimodal.csv`
- `test_multimodal.csv`
- `healthy_train_only.csv`

The notebook confirms that the dataset was fused, normalized, and split successfully, with healthy control samples isolated for the autoencoder pretraining phase.

5.8 FastAPI and Frontend Integration

The frontend communicates with the backend through HTTP requests. When the user clicks the analysis button, the frontend sends the MRI scan information and clinical metadata to the FastAPI backend.

The general integration workflow is:

1. User uploads or selects an MRI scan.
2. User enters clinical metadata.
3. Frontend sends the request to the FastAPI endpoint.
4. Backend processes the input and runs model inference.
5. Backend returns the prediction and confidence score.
6. Frontend displays the result in the diagnostic summary panel.

The API integration allows the web interface to function as an interactive AI-assisted prototype instead of only a static design.

5.9 ngrok Deployment and Public Testing

During development, the FastAPI backend was running locally. To allow the frontend or external users to access the local backend, ngrok was used to create a secure public tunnel.

The generated ngrok domain was:

```
undepressive-esmeralda-frolicsomely.ngrok-free.dev
```

The full public API URL was:

```
https://undepressive-esmeralda-frolicsomely.ngrok-free.dev
```

ngrok was useful because it allowed the local FastAPI backend to be tested through a public HTTPS URL without requiring a permanent cloud server. This helped test the connection between the frontend and backend during development.

5.10 Explainability Implementation

The system was designed to support explainability through Grad-CAM-style heatmap visualization. The purpose of this feature is to make the AI output more interpretable by showing which MRI regions contributed most strongly to the prediction.

In the interface, the heatmap can be displayed over MRI slices to help users visually inspect the important regions. This supports the clinical trust requirement because the user can compare the original MRI preview with the highlighted activation regions.

Grad-CAM support is consistent with the methodology described earlier, where heatmaps can be overlaid on MRI slices to highlight anatomical regions that influence the model prediction, such as the hippocampus, prefrontal cortex, and anterior cingulate cortex.

In the implemented backend, the Grad-CAM heatmap is generated during inference by registering hooks on the MRI encoder layer, computing gradients, resizing the activation map back to the original MRI dimensions, and returning the final heatmap as a Base64-encoded NIfTI file.

5.11 System Workflow

The implemented system follows a linear workflow:

1. **Open the web interface**
   The user opens the MRI depression detection prototype.

2. **Upload MRI scan**
   The user uploads or selects a T1-weighted MRI scan.
3. **Enter clinical metadata**
   The user provides age, sex, education, and HAMD score.
4. **Run AI analysis**
   The frontend sends the input to the FastAPI backend.
5. **Model inference**
   The backend prepares the input and runs the trained multimodal model.
6. **Display results**
   The interface displays the predicted class, confidence score, MRI preview, and heatmap visualization.
7. **Start new scan**
   The user can reset the workflow and analyze another case.

5.12 Implementation Challenges

Several challenges were encountered during system implementation.

First, connecting the frontend interface with the AI backend required separating the system into a web layer and an API layer. The frontend was responsible for user interaction, while the backend was responsible for model inference.

Second, MRI data are large and require special handling compared with normal image files. Therefore, the system needed preprocessing steps such as resizing, normalization, and conversion into tensors before inference.

Third, deploying the backend for testing required a public URL. Since the backend was initially running locally, ngrok was used to expose the FastAPI server through a secure public domain.

Finally, the system had to clearly present the result as research support only. This was important because the model output should not be interpreted as a final medical diagnosis.

5.13 Chapter Summary

This chapter described the implementation of the AI-assisted depression detection system. The system was implemented using a Next.js and React frontend, a FastAPI backend, and a PyTorch-based multimodal deep learning model.

The frontend provides the user interface for MRI upload, clinical metadata entry, analysis execution, and result visualization. The backend handles API requests, input preparation, model inference, and response generation. The trained model combines 3D MRI features with clinical metadata using a late-fusion architecture supported by Squeeze-and-Excitation attention and autoencoder-based healthy brain pretraining.

The system was also tested using ngrok, which provided a public HTTPS URL for connecting the local FastAPI backend with the frontend during development. Overall, the implementation successfully transforms the system design into a working prototype for AI-assisted depression detection from brain MRI.

CHAPTER 6: DEVELOPMENT OF THE MULTIMODAL MDD DETECTION MODEL

6.1 Introduction

This chapter presents the development of the proposed multimodal deep learning model for detecting Major Depressive Disorder (MDD) using structural 3D brain MRI scans and clinical metadata. Unlike previous chapters that focused on methodology, system design, and implementation flow, this chapter focuses specifically on the deep learning model architecture and its training strategy.

The proposed model is designed as a SOTA-inspired multimodal late-fusion framework. It combines two complementary sources of information: anatomical features extracted from T1-weighted 3D structural MRI volumes and clinical variables such as sex, age, education, and HAMD score. The model uses a 3D convolutional imaging branch, Squeeze-and-Excitation attention, a clinical metadata branch based on a Multi-Layer Perceptron, and a final binary classification head. The project notebook describes this architecture as a multimodal framework that integrates MRI-based anatomical features with clinical variables for MDD detection.

The main goal of this model is to classify each subject as either Healthy Control or MDD while preserving the spatial information of the full 3D MRI volume.

6.2 Model Development Overview

The development of the model follows five main stages:

1. Multimodal data preparation
2. 3D MRI dataset loading and augmentation
3. Unsupervised healthy-brain autoencoder pretraining
4. Supervised multimodal late-fusion classification
5. Independent test-set evaluation

The model does not rely only on MRI scans. Instead, it combines imaging data and clinical metadata. This design is important because MDD is not only reflected in brain structure but also in clinical indicators such as symptom severity and demographic information.

6.3 Input Modalities

6.3.1 MRI Input

The imaging input consists of preprocessed T1-weighted structural MRI scans. Each MRI scan is loaded as a 3D numerical volume and resized to a fixed shape of:

64 × 64 × 64

The MRI input is represented as a 3D tensor with one channel:

1 × 64 × 64 × 64

This allows the model to process the brain as a full volumetric structure rather than separate 2D slices. Using 3D input helps preserve spatial relationships between anatomical regions.

6.3.2 Clinical Metadata Input

The clinical input consists of four patient-level variables:

Feature	Description
Sex	Biological sex encoded numerically
Age	Subject age
Education	Years or level of education
HAMD	Hamilton Depression Rating Scale score

Table 6.1: Clinical Metadata Input

For Healthy Control subjects, the HAMD score is assigned as 0.0. Continuous variables are normalized using Z-score normalization to make their scale suitable for neural network training.

6.4 Multimodal Dataset Engine

A custom PyTorch dataset class is used to load both MRI volumes and clinical metadata. This dataset engine performs several important operations before passing the data into the model.

First, it loads the MRI volume from the saved `.npy` file. Then, it applies dynamic brain cropping to reduce unnecessary background regions. After that, the volume is resized to the fixed input size of 64 × 64 × 64 using trilinear interpolation.

The dataset engine also applies optional medical-style augmentations during training, including:

- 3D cutout
- Gaussian noise
- Gamma intensity adjustment
- Z-axis flipping

These augmentations help improve generalization and reduce overfitting. Finally, the MRI volume is normalized using Z-score normalization based on non-zero brain voxels.

The dataset supports two modes:

Mode	Purpose
Autoencoder mode	Returns only MRI volumes for unsupervised pretraining
Full mode	Returns MRI volume, clinical metadata, and diagnostic label

Table 6.2: Dataset Modes and Functional Purposes

This design allows the same dataset engine to be used for both healthy-brain pretraining and supervised MDD classification.

6.5 Normative Anatomy Pretraining Using 3D Autoencoder

Before training the final classifier, a 3D convolutional autoencoder is trained using only Healthy Control MRI scans. The purpose of this stage is to learn a normative representation of healthy brain anatomy.

The autoencoder consists of two parts:

Component	Function
Encoder	Compresses the MRI volume into latent anatomical features
Decoder	Reconstructs the MRI volume from the compressed representation

Table 6.3: Autoencoder Structural Units and Their Functions

The encoder learns general anatomical patterns from healthy brains. After training, the encoder weights are saved as:

blueprint.pth

This saved encoder is later loaded into the final classification model. Therefore, the final model does not start from random weights. Instead, it begins with a learned healthy-brain anatomical representation.

The notebook shows that the autoencoder was trained using Mean Squared Error loss, AdamW optimizer, and 15 epochs. The reconstruction error decreased from 0.4939 in the first epoch to approximately 0.1200 by the final epoch, indicating that the autoencoder successfully learned to reconstruct healthy MRI volumes.

6.6 MRI Imaging Branch

The MRI imaging branch is responsible for extracting anatomical features from the 3D MRI volume. It uses a sequence of 3D convolutional layers.

The architecture of the MRI encoder is:

Layer	Input Channels	Output Channels	Operation
Conv3D Block 1	1	16	3D convolution + BatchNorm + ReLU
Conv3D Block 2	16	32	3D convolution + BatchNorm + ReLU
Conv3D Block 3	32	64	3D convolution + BatchNorm + ReLU

Conv3D Block 4	64	128	3D convolution + BatchNorm + ReLU

Table 6.4: Architectural Details of the MRI Imaging Branch Layers

Each convolution uses stride 2, which gradually reduces the spatial size of the MRI volume while increasing the number of feature channels. This allows the model to learn low-level features in early layers and more abstract anatomical patterns in deeper layers.

The notebook defines this branch inside the `MultimodalFusionNetwork`, where the image encoder uses four 3D convolutional blocks with output channels 16, 32, 64, and 128.

6.7 Squeeze-and-Excitation Attention Block

After the MRI encoder extracts feature maps, a Squeeze-and-Excitation block is applied. This attention mechanism helps the model decide which feature channels are more important.

The SE block works in two steps:

1. **Squeeze step:**
   Global average pooling summarizes each feature channel into a single value.
2. **Excitation step:**
   A small fully connected network learns channel weights and applies them back to the feature maps.

This means the model can emphasize useful anatomical feature channels and reduce the effect of less informative channels.

In this project, the SE attention block is applied after the final 3D convolutional layer, where the MRI branch produces 128 feature channels. This improves the model's ability to focus on discriminative MRI features before fusion with clinical metadata.

6.8 Clinical Metadata Branch

The clinical branch processes the non-imaging variables. Since the clinical input is tabular data, it is processed using a Multi-Layer Perceptron.

The clinical branch receives four input features:

Sex, Age, Education, HAMD

The architecture is:

Layer	Input Size	Output Size	Operation
Linear 1	4	16	Fully connected + BatchNorm + ReLU
Linear 2	16	32	Fully connected + BatchNorm + ReLU

Table 6.5: Multi-Layer Perceptron Architecture for the Clinical Metadata Branch

The output of this branch is a 32-dimensional clinical feature vector. This vector represents patient-level information that may help the model improve classification beyond MRI features alone.

6.9 Late-Fusion Strategy

The model uses a late-fusion strategy. This means MRI features and clinical features are processed separately first, then combined at a later stage.

The MRI branch produces a 128-dimensional feature vector.
The clinical branch produces a 32-dimensional feature vector.

These two vectors are concatenated into one fused representation:

128 + 32 = 160 features

This fused vector is then passed into the final classification head.

Late fusion is suitable for this project because MRI data and clinical metadata are different types of data. MRI volumes are spatial and high-dimensional, while clinical metadata are compact tabular variables. Processing them separately before fusion allows each branch to learn modality-specific features.

The notebook describes the final architecture as a late-fusion model that combines a 3D CNN imaging branch with a clinical MLP branch, then concatenates the extracted representations before classification.

6.10 Classification Head

The classification head receives the 160-dimensional fused representation and predicts the probability of MDD.

The classifier architecture is:

Layer	Input Size	Output Size	Operation
Linear 1	160	64	Fully connected + BatchNorm + ReLU
Dropout	64	64	Regularization
Linear 2	64	1	Binary output logit

Table 6.6: Architectural Specifications of the Classification Head

The final output is a single logit. During inference, the sigmoid function is applied to convert this logit into a probability between 0 and 1.

The binary prediction is then made using a threshold of 0.5:

Probability	Predicted Class
< 0.5	Healthy Control
≥ 0.5	MDD

Table 6.7: Decision Threshold and Classification Criteria

6.11 Training Strategy

The supervised multimodal model is trained using both MRI volumes and clinical metadata. The training process uses several advanced techniques to improve robustness and reduce overfitting.

6.11.1 Loss Function

The model uses sigmoid focal loss. Focal loss is useful when there is class imbalance because it gives more attention to difficult samples and reduces the effect of easy examples.

6.11.2 Multimodal MixUp

Multimodal MixUp is applied to both MRI inputs and clinical metadata at the same time. This means two samples are mixed together using the same mixing ratio.

The model therefore learns smoother decision boundaries and becomes less sensitive to individual noisy samples.

6.11.3 Label Smoothing

Label smoothing is used to reduce overconfidence. Instead of forcing the model to learn labels as exactly 0 or 1, the target labels are slightly softened.

This helps the model generalize better and reduces the risk of overfitting.

6.11.4 Optimizer

The model is optimized using AdamW with weight decay. AdamW is suitable for deep learning models because it combines adaptive learning rates with better regularization.

6.11.5 Learning Rate Scheduler

A Cosine Annealing Warm Restarts scheduler is used to adjust the learning rate during training. This helps the model escape poor local minima and improve convergence.

6.11.6 Gradient Clipping

Gradient clipping is applied to prevent unstable updates. This is especially useful when training 3D CNNs because they are computationally heavy and can produce large gradients.

The notebook states that the training protocol uses multimodal MixUp, label smoothing, focal loss, AdamW optimization, gradient clipping, and cosine annealing learning-rate scheduling to improve robustness and generalization.

6.12 Model Checkpointing

During training, the model is evaluated on the validation set after each epoch. If the validation accuracy improves, the model weights are saved as:

best_multimodal_model.pth

This ensures that the final selected model is the one that performs best on validation data, not necessarily the one from the last epoch.

This checkpoint is later loaded for independent test-set evaluation.

6.13 Model Evaluation

After training, the best saved model is evaluated on the independent test set. The test set was isolated during the data preparation stage and was not used during training or validation.

The evaluation process includes:

- Loading the test dataset
- Rebuilding the same model architecture
- Loading `best_multimodal_model.pth`
- Running inference without gradient calculation
- Applying sigmoid activation to obtain probabilities
- Converting probabilities into binary predictions
- Computing diagnostic metrics

The notebook reports the following independent test-set results:

Metric	Result
Accuracy	97%
Test AUC	0.9870
Sensitivity / Recall	0.9440
Specificity	1.0000

Table 6.8: Model Performance Metrics on the Independent Test Set

	Predicted Healthy	Predicted MDD
Actual Healthy	92	0
Actual MDD	7	118

Table 6.9: Confusion Matrix for Predicted vs. Actual Classes

These results indicate strong diagnostic performance on the independent test set. However, because this is an academic prototype, the model should be interpreted as a research decision-support system, not as a certified clinical diagnostic tool.

6.14 Final Model Architecture Summary

The complete model architecture can be summarized as follows:

Component	Description
Input 1	3D T1-weighted MRI volume
Input 2	Clinical metadata: sex, age, education, HAMD
Pretraining	3D autoencoder trained on Healthy Control subjects
MRI Branch	3D CNN encoder initialized with healthy-brain blueprint
Attention	Squeeze-and-Excitation block
Clinical Branch	MLP for tabular clinical variables
Fusion Method	Late fusion by feature concatenation
Classifier	Fully connected binary classification head
Output	Healthy Control or MDD probability

Table 6.10: Comprehensive Summary of the Final Model Architecture Components

6.15 Chapter Summary

This chapter described the development of the proposed multimodal deep learning model for MDD detection. The model combines 3D structural MRI scans with clinical metadata using a late-fusion strategy. A 3D convolutional autoencoder is first trained on Healthy Control MRI scans to learn a normative anatomical representation. The learned encoder is then used as a blueprint for the MRI branch of the final diagnostic model.

The final architecture consists of a 3D CNN imaging branch, a Squeeze-and-Excitation attention block, a clinical metadata MLP branch, and a fused binary classification head. The model is trained using advanced techniques such as

multimodal MixUp, label smoothing, focal loss, AdamW optimization, gradient clipping, and cosine annealing scheduling.

The chapter also explained how the best model checkpoint is selected using validation performance and then evaluated on an independent test set. Overall, the developed model represents the core deep learning component of the proposed AI-assisted MDD detection framework.

NOTEBOOK : DEEP LEARNING BASED DEPRESSION DETECTION FROM BRAIN MRI

Phase 1 — Multimodal Data Engineering, Stratification, and Test Set Isolation

Objective

This cell prepares the final multimodal dataset by combining preprocessed 3D MRI volume paths with clinical metadata. The goal is to create clean training, validation, and independent test sets for the MDD classification task.

What This Cell Does

- Mounts Google Drive to access the project files.
- Loads the MDD and Healthy Control clinical metadata files.
- Extracts the required clinical features:
    - Sex
    - Age
    - Education
    - HAMD score
- Assigns a HAMD score of 0.0 for Healthy Control subjects.
- Cleans missing clinical values using:
    - Mode for categorical-like values such as Sex
    - Median for continuous variables such as Age, Education, and HAMD
- Loads the master MRI label file containing the MRI volume paths and labels.
- Extracts subject IDs from MRI file names.
- Merges MRI data with clinical metadata using subject_id.
- Applies Z-score normalization to continuous clinical variables.
- Splits the dataset into:
    - 80% training set
    - 10% validation set
    - 10% independent test set
- Isolates only Healthy Control samples from the training set for the autoencoder pretraining phase.

Expected Output

This cell produces four CSV files:

- `train_multimodal.csv`
- `val_multimodal.csv`
- `test_multimodal.csv`
- `healthy_train_only.csv`

Project Relevance

This phase is essential because it builds the final multimodal dataset used by the model. It also prevents data leakage by keeping the test set completely separate and by using only healthy training samples for the normative autoencoder pretraining phase.

```
#
============================================================
============================

# CELL 1: Multimodal Data Engineering, Stratification,
and Test Set Isolation

#
============================================================
============================

from google.colab import drive

import pandas as pd

import numpy as np

import os

import re

from sklearn.model_selection import train_test_split


drive.mount('/content/drive', force_remount=True)

PROJECT_PATH = '/content/drive/MyDrive/Colab Notebooks'

MASTER_OUTPUT_PATH = os.path.join(PROJECT_PATH,
'final_master_labels.csv')


print("Phase 1: Initiating Multimodal Data Engineering
Pipeline...\n")


mdd_filename = 'MDD.xlsx'

nc_filename = 'NC.xlsx'


mdd_path = os.path.join(PROJECT_PATH, mdd_filename)

nc_path = os.path.join(PROJECT_PATH, nc_filename)


def load_file(filepath):

    if not os.path.exists(filepath):
```

```
        raise FileNotFoundError(f"Error: File not found at {filepath}. Please ensure the filename and extension are correct.")
    if filepath.endswith('.csv'):
        return pd.read_csv(filepath)
    else:
        return pd.read_excel(filepath)

mdd_df = load_file(mdd_path)
nc_df = load_file(nc_path)

def clean_clinical_data(df, is_mdd=True):
    df_clean = pd.DataFrame()

    if 'ID' in df.columns:
        df_clean['ID'] = df['ID']
    else:
        raise KeyError("Column 'ID' was not found in the clinical file.")

    sex_col = [c for c in df.columns if 'Sex' in c or '性别' in c][0]
    age_col = [c for c in df.columns if 'Age' in c or '年龄' in c][0]
    edu_col = [c for c in df.columns if 'Education' in c or '教育' in c][0]

    df_clean['Sex'] = df[sex_col]
    df_clean['Age'] = df[age_col]
    df_clean['Education'] = df[edu_col]

    if is_mdd:
        hamd_col = [c for c in df.columns if 'HAMD' in c][0]
        df_clean['HAMD'] = df[hamd_col]
    else:
        df_clean['HAMD'] = 0.0
```

```
    return df_clean


mdd_clin = clean_clinical_data(mdd_df, is_mdd=True)
nc_clin = clean_clinical_data(nc_df, is_mdd=False)


clinical_master = pd.concat([mdd_clin, nc_clin],
ignore_index=True)


for col in ['Sex', 'Age', 'Education', 'HAMD']:
    clinical_master[col] =
pd.to_numeric(clinical_master[col], errors='coerce')


clinical_master['Sex'] =
clinical_master['Sex'].fillna(clinical_master['Sex'].mode
()[0])
clinical_master['Age'] =
clinical_master['Age'].fillna(clinical_master['Age'].medi
an())
clinical_master['Education'] =
clinical_master['Education'].fillna(clinical_master['Educ
ation'].median())
clinical_master['HAMD'] =
clinical_master['HAMD'].fillna(0.0)
clinical_master['subject_id'] =
clinical_master['ID'].astype(str).str.strip()


master_df = pd.read_csv(MASTER_OUTPUT_PATH)
if 'file_path' in master_df.columns:
    master_df = master_df.rename(columns={'file_path':
'output_npy'})


def extract_id(filepath):
    match = re.search(r'(IS\d+-\d+-\d+)',
os.path.basename(filepath))
    return match.group(1) if match else
os.path.basename(filepath)
```

```
master_df['subject_id'] =
master_df['output_npy'].apply(extract_id).astype(str).str
.strip()
fused_df = pd.merge(master_df, clinical_master,
on='subject_id', how='left')


for col in ['Sex', 'Age', 'Education', 'HAMD']:
    if col == 'Sex':
        fused_df[col] =
fused_df[col].fillna(clinical_master['Sex'].mode()[0])
    else:
        fused_df[col] =
fused_df[col].fillna(clinical_master[col].median())


for col in ['Age', 'Education', 'HAMD']:
    fused_df[col] = (fused_df[col] -
fused_df[col].mean()) / (fused_df[col].std() + 1e-8)


# ------------------------------------------------------
----------------------
# Split into Train / Validation / Test
# 80% Train, 10% Validation, 10% Test
# ------------------------------------------------------
----------------------
train_df, temp_df = train_test_split(
    fused_df,
    test_size=0.2,
    stratify=fused_df['label'],
    random_state=42
)


val_df, test_df = train_test_split(
    temp_df,
    test_size=0.5,
    stratify=temp_df['label'],
    random_state=42
)
```

```
# Only healthy controls from training set are used for
autoencoder pretraining
healthy_train_df = train_df[train_df['label'] ==
0].copy()


train_df.to_csv(os.path.join(PROJECT_PATH,
'train_multimodal.csv'), index=False)
val_df.to_csv(os.path.join(PROJECT_PATH,
'val_multimodal.csv'), index=False)
test_df.to_csv(os.path.join(PROJECT_PATH,
'test_multimodal.csv'), index=False)
healthy_train_df.to_csv(os.path.join(PROJECT_PATH,
'healthy_train_only.csv'), index=False)


print("Phase 1 Complete: Dataset successfully fused,
normalized, and split.")
print(f"Training samples: {len(train_df)}")
print(f"Validation samples: {len(val_df)}")
print(f"Test samples: {len(test_df)}")
print(f"Normative (Healthy) training samples isolated:
{len(healthy_train_df)}")
```

Output :

```
Mounted at /content/drive
Phase 1: Initiating Multimodal Data Engineering Pipeline...


/usr/local/lib/python3.12/dist-
packages/openpyxl/worksheet/_reader.py:329: UserWarning:
Unknown extension is not supported and will be removed
  warn(msg)
/usr/local/lib/python3.12/dist-
packages/openpyxl/worksheet/_reader.py:329: UserWarning:
Unknown extension is not supported and will be removed
  warn(msg)
Phase 1 Complete: Dataset successfully fused, normalized,
and split.
Training samples: 1732
Validation samples: 217
```

```
Test samples: 217

Normative (Healthy) training samples isolated: 736
```

Phase 2 — Medical-Grade Multimodal Dataset Engine

Objective

This cell defines a custom PyTorch Dataset class that loads both 3D MRI volumes and clinical metadata for multimodal deep learning.

What This Cell Does

- Reads the dataset CSV file containing MRI paths, labels, and clinical features.
- Loads each 3D MRI volume from its `.npy` file.
- Applies dynamic brain cropping to remove unnecessary background areas.
- Resizes each MRI volume to a fixed shape of 64 × 64 × 64.
- Applies optional medical-style 3D augmentations during training, including:
  - 3D Cutout
  - Gaussian noise
  - Gamma intensity adjustment
  - Z-axis flipping
- Normalizes MRI voxel intensities using Z-score normalization.
- Supports two modes:
  - autoencoder: returns only the MRI volume for unsupervised pretraining.
  - full: returns MRI volume, clinical metadata, and label for supervised classification.

Expected Output

The cell initializes the dataset engine successfully and prints a confirmation message.

Project Relevance

This dataset class is the core data-loading component of the project. It allows the model to learn from both structural brain MRI features and clinical metadata, which supports the final multimodal MDD detection system.

```
#
============================================================
============================

# CELL 2: Medical-Grade Multimodal Dataset Engine

#
============================================================
============================

import torch
```

```
import torch.nn.functional as F
from torch.utils.data import Dataset
import numpy as np
import pandas as pd
import random


class MultimodalDatasetPro(Dataset):
    def __init__(self, csv_path, augment=False,
target_shape=(64, 64, 64), mode='full'):
        """
        Args:
            csv_path (str): Path to the dataset CSV file.
            augment (bool): Whether to apply 3D medical
augmentations.
            target_shape (tuple): Desired spatial
dimensions for the 3D MRI volumes.
            mode (str): 'full' for supervised multimodal
training, 'autoencoder' for unsupervised pre-training.
        """
        self.df = pd.read_csv(csv_path)
        self.augment = augment
        self.target_shape = target_shape
        self.mode = mode


    def _crop_brain(self, volume):
        """Dynamic 3D bounding box to isolate brain
tissue from background noise."""
        idx = np.argwhere(volume > 0.01)
        if len(idx) == 0:
            return volume
        z_m, y_m, x_m = idx.min(axis=0)
        z_M, y_M, x_M = idx.max(axis=0)
        return volume[z_m:z_M+1, y_m:y_M+1, x_m:x_M+1]
```

```
    def _medical_augmentation(self, volume):
        """Applies domain-specific, anatomically safe 3D
augmentations."""
        if random.random() > 0.5:
            d, h, w = volume.shape[1:]
            b = int(d * 0.20)
            z = random.randint(0, d - b)
            y = random.randint(0, h - b)
            x = random.randint(0, w - b)
            volume[:, z:z+b, y:y+b, x:x+b] = 0.0

        if random.random() > 0.4:
            volume += torch.randn_like(volume) *
random.uniform(0.01, 0.05)

        if random.random() > 0.4:
            volume = torch.sign(volume) *
(torch.abs(volume) ** random.uniform(0.8, 1.2))

        if random.random() > 0.5:
            volume = torch.flip(volume, dims=[1])

        return volume

    def __getitem__(self, idx):
        row = self.df.iloc[idx]

        vol =
np.load(row['output_npy']).astype(np.float32)
        vol = self._crop_brain(vol)

        vol_tensor =
torch.tensor(vol).unsqueeze(0).unsqueeze(0)
```

```
        vol = F.interpolate(vol_tensor,
size=self.target_shape, mode='trilinear',
align_corners=False).squeeze(0)


        if self.augment:
            vol = self._medical_augmentation(vol)


        mean_val = vol[vol > 0].mean()
        std_val = vol[vol > 0].std()
        if std_val > 0:
            vol = (vol - mean_val) / std_val


        if self.mode == 'autoencoder':
            return vol


        clin = torch.tensor([row['Sex'], row['Age'],
row['Education'], row['HAMD']], dtype=torch.float32)
        label = torch.tensor(float(row['label']),
dtype=torch.float32)


        return vol, clin, label


    def __len__(self):
        return len(self.df)


print("Phase 2 Complete: Medical-Grade Multimodal Dataset
Engine initialized.")
```

Output :

```
Phase 2 Complete: Medical-Grade Multimodal Dataset Engine
initialized.
```

Phase 3 — Unsupervised Normative Anatomy Pretraining Using Autoencoder

Objective

This cell trains a 3D convolutional autoencoder using only Healthy Control MRI scans. The purpose is to learn a general representation of normal brain anatomy before training the final MDD classification model.

What This Cell Does

- Loads the `healthy_train_only.csv` file created in Phase 1.
- Creates a DataLoader for Healthy Control MRI volumes.
- Defines a 3D convolutional autoencoder with:
    - An encoder that compresses the MRI volume into latent anatomical features.
    - A decoder that reconstructs the original MRI volume from the compressed representation.
- Trains the autoencoder using Mean Squared Error loss.
- Uses AdamW optimizer for stable training.
- Uses mixed precision training when CUDA is available.
- Tracks the best reconstruction loss.
- Saves the encoder weights as `blueprint.pth`.

Expected Output

The cell prints the reconstruction error for each epoch. A decreasing MSE value indicates that the autoencoder is learning to reconstruct healthy brain MRI volumes effectively.

Project Relevance

The saved encoder acts as a normative anatomical blueprint. This blueprint is later injected into the final classification model so the model starts with learned healthy-brain representations instead of learning from scratch.

```
  #
============================================================
============================
  # CELL 3: Foundation Model (Healthy Brain Pre-training)
  #
============================================================
============================

  import torch

  import torch.nn as nn
```

```
  import torch.optim as optim
  from torch.utils.data import DataLoader
  from torch.amp import autocast, GradScaler
  import os

  PROJECT_PATH = '/content/drive/MyDrive/Colab Notebooks'
  DEVICE = torch.device("cuda" if
torch.cuda.is_available() else "cpu")

  healthy_csv = os.path.join(PROJECT_PATH,
'healthy_train_only.csv')
  healthy_ds = MultimodalDatasetPro(csv_path=healthy_csv,
augment=True, mode='autoencoder')
  healthy_loader = DataLoader(healthy_ds, batch_size=8,
shuffle=True, num_workers=2)

  class BrainAutoencoder3D(nn.Module):
      def __init__(self):
          super(BrainAutoencoder3D, self).__init__()

          self.encoder = nn.Sequential(
              nn.Conv3d(1, 16, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(16), nn.ReLU(True),
              nn.Conv3d(16, 32, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(32), nn.ReLU(True),
              nn.Conv3d(32, 64, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(64), nn.ReLU(True),
              nn.Conv3d(64, 128, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(128), nn.ReLU(True)
          )

          self.decoder = nn.Sequential(
```

```
                nn.ConvTranspose3d(128, 64, kernel_size=3,
stride=2, padding=1, output_padding=1),
nn.BatchNorm3d(64), nn.ReLU(True),
                nn.ConvTranspose3d(64, 32, kernel_size=3,
stride=2, padding=1, output_padding=1),
nn.BatchNorm3d(32), nn.ReLU(True),
                nn.ConvTranspose3d(32, 16, kernel_size=3,
stride=2, padding=1, output_padding=1),
nn.BatchNorm3d(16), nn.ReLU(True),
                nn.ConvTranspose3d(16, 1, kernel_size=3,
stride=2, padding=1, output_padding=1)
            )

        def forward(self, x):
            return self.decoder(self.encoder(x))

    model_ae = BrainAutoencoder3D().to(DEVICE)
    criterion = nn.MSELoss()
    optimizer = optim.AdamW(model_ae.parameters(), lr=1e-3,
weight_decay=1e-4)
    scaler = GradScaler('cuda') if DEVICE.type == 'cuda'
else None

    EPOCHS_AE = 15
    best_mse = float('inf')

    print(f"\nPhase 3: Initiating Unsupervised Normative
Anatomy Pre-training on {DEVICE}...")

    for epoch in range(EPOCHS_AE):
        model_ae.train()
        running_loss = 0.0
```

```
        for volumes in healthy_loader:
            volumes = volumes.to(DEVICE)
            optimizer.zero_grad()

            if scaler:
                with autocast('cuda'):
                    loss = criterion(model_ae(volumes),
volumes)
                scaler.scale(loss).backward()
                scaler.step(optimizer)
                scaler.update()
            else:
                loss = criterion(model_ae(volumes),
volumes)
                loss.backward()
                optimizer.step()

            running_loss += loss.item()

        avg_loss = running_loss / len(healthy_loader)
        print(f"Epoch [{epoch+1:02d}/{EPOCHS_AE}] |
Reconstruction Error (MSE): {avg_loss:.4f}")

        if avg_loss < best_mse:
            best_mse = avg_loss
            blueprint_path = os.path.join(PROJECT_PATH,
'blueprint.pth')
            torch.save(model_ae.encoder.state_dict(),
blueprint_path)
```

```
  print("Phase 3 Complete: Normative anatomical blueprint
successfully extracted and saved.")
```

Output :

```
Phase 3: Initiating Unsupervised Normative Anatomy Pre-
training on cuda...
Epoch [01/15] | Reconstruction Error (MSE): 0.4939
Epoch [02/15] | Reconstruction Error (MSE): 0.2057
Epoch [03/15] | Reconstruction Error (MSE): 0.1695
Epoch [04/15] | Reconstruction Error (MSE): 0.1539
Epoch [05/15] | Reconstruction Error (MSE): 0.1452
Epoch [06/15] | Reconstruction Error (MSE): 0.1394
Epoch [07/15] | Reconstruction Error (MSE): 0.1337
Epoch [08/15] | Reconstruction Error (MSE): 0.1315
Epoch [09/15] | Reconstruction Error (MSE): 0.1282
Epoch [10/15] | Reconstruction Error (MSE): 0.1258
Epoch [11/15] | Reconstruction Error (MSE): 0.1226
Epoch [12/15] | Reconstruction Error (MSE): 0.1205
Epoch [13/15] | Reconstruction Error (MSE): 0.1213
Epoch [14/15] | Reconstruction Error (MSE): 0.1185
Epoch [15/15] | Reconstruction Error (MSE): 0.1200
Phase 3 Complete: Normative anatomical blueprint
successfully extracted and saved.
```

Phase 4 — State-of-the-Art (SOTA)-Inspired Multimodal Late-Fusion Network

Objective

This cell trains the main diagnostic model for MDD classification using both 3D MRI features and clinical metadata.

What This Cell Does

- Loads the training and validation multimodal CSV files.
- Creates PyTorch DataLoaders for training and validation.
- Defines a 3D Squeeze-and-Excitation block to improve channel-wise feature attention.

- Defines the final multimodal fusion network with two branches:
    - MRI branch: extracts 3D imaging features using a convolutional encoder.
    - Clinical branch: extracts tabular metadata features using a small MLP.
- Loads the pretrained healthy-brain encoder weights from `blueprint.pth`.
- Concatenates MRI and clinical features using a late-fusion strategy.
- Passes the fused representation into a classifier head.
- Applies training techniques to improve generalization:
    - Multimodal MixUp
    - Label smoothing
    - Focal Loss
    - Gradient clipping
    - AdamW optimizer
    - Cosine Annealing Warm Restarts scheduler
- Saves the best model as `best_multimodal_model.pth` based on validation accuracy.

Expected Output

The cell prints training accuracy and validation accuracy for each epoch. Whenever validation accuracy improves, the model checkpoint is saved.

Project Relevance

This is the main training phase of the project. It combines MRI-based deep features with clinical metadata to improve MDD classification performance and create the final diagnostic model.

```
#
============================================================
============================
# CELL 4: State-of-the-Art (SOTA)-Inspired Multimodal
Late-Fusion Network
#
============================================================
============================
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torch.amp import autocast, GradScaler
import torchvision
import pandas as pd
import numpy as np
```

```
import os


PROJECT_PATH = '/content/drive/MyDrive/Colab Notebooks'
DEVICE = torch.device("cuda" if torch.cuda.is_available()
else "cpu")


train_df = pd.read_csv(os.path.join(PROJECT_PATH,
'train_multimodal.csv'))
alpha_val = (len(train_df) - train_df['label'].sum()) /
len(train_df)


train_ds =
MultimodalDatasetPro(csv_path=os.path.join(PROJECT_PATH,
'train_multimodal.csv'), augment=True, mode='full')
val_ds =
MultimodalDatasetPro(csv_path=os.path.join(PROJECT_PATH,
'val_multimodal.csv'), augment=False, mode='full')


train_loader = DataLoader(train_ds, batch_size=8,
shuffle=True, num_workers=2)
val_loader = DataLoader(val_ds, batch_size=8,
shuffle=False, num_workers=2)


class SEBlock3D(nn.Module):
    """Squeeze-and-Excitation mechanism for 3D channel
attention."""
    def __init__(self, channel, reduction=16):
        super(SEBlock3D, self).__init__()
        self.avg_pool = nn.AdaptiveAvgPool3d(1)
        self.fc = nn.Sequential(
            nn.Linear(channel, channel // reduction,
bias=False), nn.ReLU(True),
            nn.Linear(channel // reduction, channel,
bias=False), nn.Sigmoid()
        )


    def forward(self, x):
        b, c, _, _, _ = x.size()
```

```
        y = self.fc(self.avg_pool(x).view(b, c)).view(b,
c, 1, 1, 1)
        return x * y.expand_as(x)


class MultimodalFusionNetwork(nn.Module):
    """Late-fusion architecture combining 3D MRI features
and clinical metadata."""
    def __init__(self):
        super(MultimodalFusionNetwork, self).__init__()


        self.img_encoder = nn.Sequential(
            nn.Conv3d(1, 16, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(16), nn.ReLU(True),
            nn.Conv3d(16, 32, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(32), nn.ReLU(True),
            nn.Conv3d(32, 64, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(64), nn.ReLU(True),
            nn.Conv3d(64, 128, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(128), nn.ReLU(True)
        )


        blueprint_path = os.path.join(PROJECT_PATH,
'blueprint.pth')
        if os.path.exists(blueprint_path):

self.img_encoder.load_state_dict(torch.load(blueprint_pat
h))
            print("System Log: Normative Anatomy
Blueprint Successfully Injected.")


        self.img_attention = SEBlock3D(128)
        self.img_flatten = nn.Sequential(nn.Flatten(),
nn.Linear(128 * 4 * 4 * 4, 128), nn.ReLU(True))


        self.clin_encoder = nn.Sequential(
            nn.Linear(4, 16), nn.BatchNorm1d(16),
nn.ReLU(True),
            nn.Linear(16, 32), nn.BatchNorm1d(32),
nn.ReLU(True)
```

```
        )

        self.classifier = nn.Sequential(
            nn.Linear(160, 64), nn.BatchNorm1d(64),
nn.ReLU(True), nn.Dropout(0.5),
            nn.Linear(64, 1)
        )

    def forward(self, img, clin):
        img_features =
self.img_flatten(self.img_attention(self.img_encoder(img)
))
        clin_features = self.clin_encoder(clin)

        fused_representation = torch.cat((img_features,
clin_features), dim=1)
        return self.classifier(fused_representation)

def mixup_multimodal(x_img, x_clin, y, alpha=0.2):
    """Applies MixUp augmentation synchronously across
multimodal inputs."""
    lam = np.random.beta(alpha, alpha) if alpha > 0 else
1
    idx = torch.randperm(x_img.size(0)).to(DEVICE)
    mixed_img = lam * x_img + (1 - lam) * x_img[idx]
    mixed_clin = lam * x_clin + (1 - lam) * x_clin[idx]
    return mixed_img, mixed_clin, y, y[idx], lam

def smooth_labels(targets, smoothing=0.1):
    """Applies label smoothing to penalize
overconfidence."""
    return targets * (1.0 - smoothing) + 0.5 * smoothing

model = MultimodalFusionNetwork().to(DEVICE)
optimizer = optim.AdamW(model.parameters(), lr=1e-4,
weight_decay=1e-3)
```

```
scheduler =
optim.lr_scheduler.CosineAnnealingWarmRestarts(optimizer,
T_0=15, T_mult=2)
scaler = GradScaler('cuda') if DEVICE.type == 'cuda' else
None


EPOCHS = 60
best_val_acc = 0.0


print(f"\nPhase 4: Commencing SOTA Multimodal Training
Protocol on {DEVICE}...")


for epoch in range(EPOCHS):
    model.train()
    train_loss, correct_train, total_train = 0, 0, 0


    for x_img, x_clin, y in train_loader:
        x_img, x_clin, y = x_img.to(DEVICE),
x_clin.to(DEVICE), y.to(DEVICE).float().view(-1)


        m_img, m_clin, y_a, y_b, lam =
mixup_multimodal(x_img, x_clin, y)
        s_y_a, s_y_b = smooth_labels(y_a),
smooth_labels(y_b)


        optimizer.zero_grad()


        if scaler:
            with autocast('cuda'):
                logits = model(m_img, m_clin).view(-1)
                loss = lam *
torchvision.ops.sigmoid_focal_loss(logits, s_y_a,
alpha=alpha_val, gamma=2., reduction='mean') + \
                       (1 - lam) *
torchvision.ops.sigmoid_focal_loss(logits, s_y_b,
alpha=alpha_val, gamma=2., reduction='mean')
            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)
```

```
torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
            scaler.step(optimizer)
            scaler.update()
        else:
            logits = model(m_img, m_clin).view(-1)
            loss = lam *
torchvision.ops.sigmoid_focal_loss(logits, s_y_a,
alpha=alpha_val, gamma=2., reduction='mean') + \
                   (1 - lam) *
torchvision.ops.sigmoid_focal_loss(logits, s_y_b,
alpha=alpha_val, gamma=2., reduction='mean')
            loss.backward()

torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
            optimizer.step()


        train_loss += loss.item()
        preds = torch.round(torch.sigmoid(logits))
        correct_train += (lam * (preds == y_a).float() +
(1 - lam) * (preds == y_b).float()).sum().item()
        total_train += y.size(0)


    model.eval()
    val_loss, correct_val, total_val = 0, 0, 0
    with torch.no_grad():
        for x_img, x_clin, y in val_loader:
            x_img, x_clin, y = x_img.to(DEVICE),
x_clin.to(DEVICE), y.to(DEVICE).float().view(-1)


            if scaler:
                with autocast('cuda'):
                    logits = model(x_img, x_clin).view(-
1)
            else:
                logits = model(x_img, x_clin).view(-1)
```

```
            val_loss +=
torchvision.ops.sigmoid_focal_loss(logits, y,
alpha=alpha_val, gamma=2., reduction='mean').item()
            correct_val +=
(torch.round(torch.sigmoid(logits)) == y).sum().item()
            total_val += y.size(0)

    train_acc = (correct_train / total_train) * 100
    val_acc = (correct_val / total_val) * 100
    scheduler.step()

    print(f"Epoch [{epoch+1:02d}/{EPOCHS}] | Train Acc:
{train_acc:.1f}% | Val Acc: {val_acc:.1f}%")

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(),
os.path.join(PROJECT_PATH, 'best_multimodal_model.pth'))
        print(f"    ★ New High Score:
{best_val_acc:.1f}%")

print("Phase 4 Complete: Multimodal Training Protocol
finished.")
```

Output :

```
System Log: Normative Anatomy Blueprint Successfully
Injected.


Phase 4: Commencing SOTA Multimodal Training Protocol on
cuda...
Epoch [01/60] | Train Acc: 54.2% | Val Acc: 77.9%
    ★ New High Score: 77.9%
Epoch [02/60] | Train Acc: 68.1% | Val Acc: 86.2%
    ★ New High Score: 86.2%
Epoch [03/60] | Train Acc: 75.1% | Val Acc: 89.9%
    ★ New High Score: 89.9%
```

```
Epoch [04/60] | Train Acc: 79.6% | Val Acc: 87.1%
Epoch [05/60] | Train Acc: 80.8% | Val Acc: 90.3%
   ★ New High Score: 90.3%
Epoch [06/60] | Train Acc: 80.3% | Val Acc: 90.3%
Epoch [07/60] | Train Acc: 81.3% | Val Acc: 90.8%
   ★ New High Score: 90.8%
Epoch [08/60] | Train Acc: 81.6% | Val Acc: 91.2%
   ★ New High Score: 91.2%
Epoch [09/60] | Train Acc: 83.6% | Val Acc: 91.2%
Epoch [10/60] | Train Acc: 83.7% | Val Acc: 92.2%
   ★ New High Score: 92.2%
Epoch [11/60] | Train Acc: 83.8% | Val Acc: 89.9%
Epoch [12/60] | Train Acc: 84.0% | Val Acc: 91.2%
Epoch [13/60] | Train Acc: 85.7% | Val Acc: 91.2%
Epoch [14/60] | Train Acc: 85.1% | Val Acc: 90.8%
Epoch [15/60] | Train Acc: 84.0% | Val Acc: 90.8%
Epoch [16/60] | Train Acc: 86.1% | Val Acc: 90.3%
Epoch [17/60] | Train Acc: 84.8% | Val Acc: 92.2%
Epoch [18/60] | Train Acc: 85.4% | Val Acc: 90.8%
Epoch [19/60] | Train Acc: 86.1% | Val Acc: 92.2%
Epoch [20/60] | Train Acc: 86.1% | Val Acc: 86.6%
Epoch [21/60] | Train Acc: 87.6% | Val Acc: 91.7%
Epoch [22/60] | Train Acc: 87.3% | Val Acc: 88.9%
Epoch [23/60] | Train Acc: 86.4% | Val Acc: 89.9%
Epoch [24/60] | Train Acc: 89.6% | Val Acc: 91.2%
Epoch [25/60] | Train Acc: 88.0% | Val Acc: 89.4%
Epoch [26/60] | Train Acc: 89.1% | Val Acc: 89.9%
Epoch [27/60] | Train Acc: 87.2% | Val Acc: 90.3%
Epoch [28/60] | Train Acc: 90.0% | Val Acc: 88.5%
Epoch [29/60] | Train Acc: 89.7% | Val Acc: 88.5%
```

```
Epoch [30/60] | Train Acc: 91.0% | Val Acc: 85.7%
Epoch [31/60] | Train Acc: 91.2% | Val Acc: 86.6%
Epoch [32/60] | Train Acc: 90.9% | Val Acc: 85.7%
Epoch [33/60] | Train Acc: 90.9% | Val Acc: 87.6%
Epoch [34/60] | Train Acc: 91.6% | Val Acc: 86.6%
Epoch [35/60] | Train Acc: 90.8% | Val Acc: 88.5%
Epoch [36/60] | Train Acc: 91.6% | Val Acc: 88.9%
Epoch [37/60] | Train Acc: 91.2% | Val Acc: 86.2%
Epoch [38/60] | Train Acc: 92.4% | Val Acc: 85.3%
Epoch [39/60] | Train Acc: 92.3% | Val Acc: 87.1%
Epoch [40/60] | Train Acc: 93.3% | Val Acc: 87.1%
Epoch [41/60] | Train Acc: 92.3% | Val Acc: 88.5%
Epoch [42/60] | Train Acc: 93.2% | Val Acc: 87.1%
Epoch [43/60] | Train Acc: 91.1% | Val Acc: 86.2%
Epoch [44/60] | Train Acc: 92.8% | Val Acc: 88.0%
Epoch [45/60] | Train Acc: 92.4% | Val Acc: 86.6%
Epoch [46/60] | Train Acc: 90.6% | Val Acc: 88.5%
Epoch [47/60] | Train Acc: 90.9% | Val Acc: 86.6%
Epoch [48/60] | Train Acc: 91.4% | Val Acc: 85.3%
Epoch [49/60] | Train Acc: 91.8% | Val Acc: 86.6%
Epoch [50/60] | Train Acc: 90.1% | Val Acc: 85.7%
Epoch [51/60] | Train Acc: 91.4% | Val Acc: 87.6%
Epoch [52/60] | Train Acc: 90.3% | Val Acc: 87.1%
Epoch [53/60] | Train Acc: 91.4% | Val Acc: 86.6%
Epoch [54/60] | Train Acc: 92.1% | Val Acc: 85.3%
Epoch [55/60] | Train Acc: 91.9% | Val Acc: 85.3%
Epoch [56/60] | Train Acc: 93.4% | Val Acc: 86.6%
Epoch [57/60] | Train Acc: 91.6% | Val Acc: 87.6%
Epoch [58/60] | Train Acc: 92.5% | Val Acc: 86.6%
Epoch [59/60] | Train Acc: 93.0% | Val Acc: 86.6%
```

```
Epoch [60/60] | Train Acc: 91.9% | Val Acc: 87.6%

Phase 4 Complete: Multimodal Training Protocol finished.
```

Phase 5 — Independent Test Set Evaluation and Diagnostic Performance Metrics

Objective

This cell evaluates the best trained multimodal model on the independent test set. The goal is to measure the final diagnostic performance of the model on unseen data.

What This Cell Does

- Loads the independent test dataset from `test_multimodal.csv`.
- Rebuilds the same model architecture used during training.
- Loads the best saved model weights from `best_multimodal_model.pth`.
- Runs inference on the test set without gradient calculation.
- Converts model logits into probabilities using the sigmoid function.
- Converts probabilities into binary predictions using a threshold of 0.5.
- Computes the confusion matrix.
- Generates the ROC curve.
- Calculates the Area Under the Curve value.
- Prints a classification report containing:
    - Precision
    - Recall
    - F1-score
    - Support
- Calculates clinical evaluation metrics:
    - Sensitivity
    - Specificity

Expected Output

This cell outputs:

- Confusion Matrix
- ROC Curve
- Test AUC
- Classification Report
- Sensitivity
- Specificity

Project Relevance

This phase provides the final evaluation of the trained system. Since the test set was isolated earlier and not used during training, these results provide an initial estimate of the model's generalization performance on unseen data.

```
#
============================================================
============================

# CELL 5: Independent Test Set Evaluation and Diagnostic
Performance Metrics

#
============================================================
============================

import matplotlib.pyplot as plt

from sklearn.metrics import confusion_matrix, roc_curve,
auc, classification_report

import torch

import numpy as np


PROJECT_PATH = '/content/drive/MyDrive/Colab Notebooks'

DEVICE = torch.device("cuda" if torch.cuda.is_available()
else "cpu")


# Load test set

test_ds = MultimodalDatasetPro(

    csv_path=os.path.join(PROJECT_PATH,
'test_multimodal.csv'),

    augment=False,

    mode='full'

)

test_loader = DataLoader(test_ds, batch_size=8,
shuffle=False, num_workers=2)


# Rebuild the same model architecture used in training

class SEBlock3D(nn.Module):

    def __init__(self, channel, reduction=16):

        super(SEBlock3D, self).__init__()
```

```
        self.avg_pool = nn.AdaptiveAvgPool3d(1)
        self.fc = nn.Sequential(
            nn.Linear(channel, channel // reduction,
bias=False),
            nn.ReLU(True),
            nn.Linear(channel // reduction, channel,
bias=False),
            nn.Sigmoid()
        )

    def forward(self, x):
        b, c, _, _, _ = x.size()
        y = self.fc(self.avg_pool(x).view(b, c)).view(b,
c, 1, 1, 1)
        return x * y.expand_as(x)

class MultimodalFusionNetwork(nn.Module):
    def __init__(self):
        super(MultimodalFusionNetwork, self).__init__()

        self.img_encoder = nn.Sequential(
            nn.Conv3d(1, 16, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(16), nn.ReLU(True),
            nn.Conv3d(16, 32, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(32), nn.ReLU(True),
            nn.Conv3d(32, 64, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(64), nn.ReLU(True),
            nn.Conv3d(64, 128, kernel_size=3, stride=2,
padding=1), nn.BatchNorm3d(128), nn.ReLU(True)
        )

        self.img_attention = SEBlock3D(128)
        self.img_flatten = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4 * 4, 128),
```

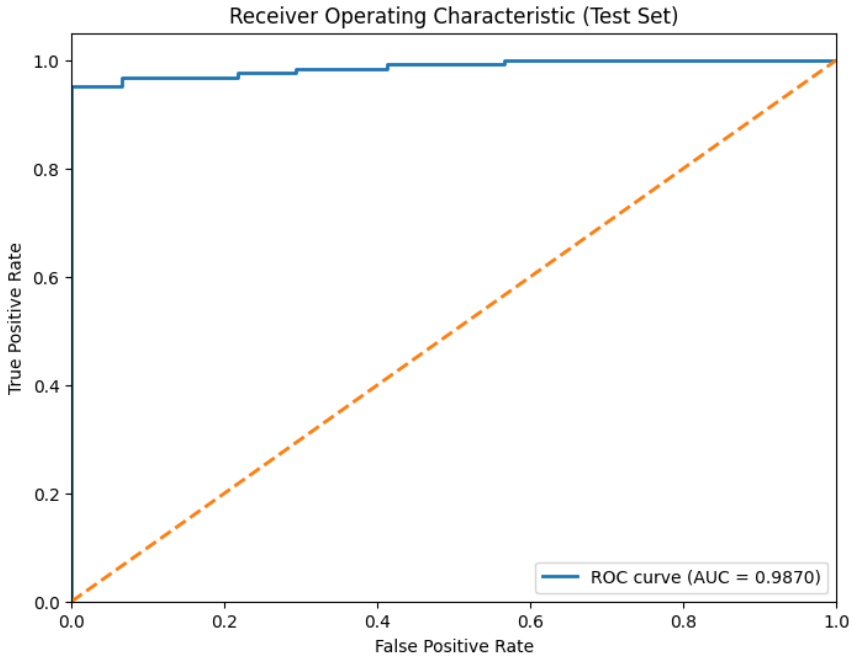
```
            nn.ReLU(True)
        )

        self.clin_encoder = nn.Sequential(
            nn.Linear(4, 16), nn.BatchNorm1d(16),
nn.ReLU(True),
            nn.Linear(16, 32), nn.BatchNorm1d(32),
nn.ReLU(True)
        )

        self.classifier = nn.Sequential(
            nn.Linear(160, 64), nn.BatchNorm1d(64),
nn.ReLU(True), nn.Dropout(0.5),
            nn.Linear(64, 1)
        )

    def forward(self, img, clin):
        img_features =
self.img_flatten(self.img_attention(self.img_encoder(img)
))
        clin_features = self.clin_encoder(clin)
        fused_representation = torch.cat((img_features,
clin_features), dim=1)
        return self.classifier(fused_representation)

# Load best trained model
model = MultimodalFusionNetwork().to(DEVICE)
model.load_state_dict(torch.load(os.path.join(PROJECT_PAT
H, 'best_multimodal_model.pth'), map_location=DEVICE))
model.eval()

all_preds = []
all_labels = []
all_probs = []
```

```
print("Phase 5: Generating Final Test Reports...\n")

with torch.no_grad():
    for x_img, x_clin, y in test_loader:
        x_img = x_img.to(DEVICE)
        x_clin = x_clin.to(DEVICE)
        y = y.to(DEVICE)

        outputs = model(x_img, x_clin).view(-1)
        probs = torch.sigmoid(outputs)
        preds = torch.round(probs)

        all_probs.extend(probs.cpu().numpy())
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(y.cpu().numpy())

all_probs = np.array(all_probs)
all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

# Confusion Matrix
cm = confusion_matrix(all_labels, all_preds)
print("Confusion Matrix:")
print(cm)

# ROC Curve + AUC
fpr, tpr, _ = roc_curve(all_labels, all_probs)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, lw=2, label=f'ROC curve (AUC =
{roc_auc:.4f})')
plt.plot([0, 1], [0, 1], lw=2, linestyle='--')
```

```
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.title('Receiver Operating Characteristic (Test Set)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.show()


print("\nClassification Report:")
print(classification_report(all_labels, all_preds,
target_names=['Healthy', 'MDD']))


# Specificity
tn, fp, fn, tp = cm.ravel()
specificity = tn / (tn + fp + 1e-8)
sensitivity = tp / (tp + fn + 1e-8)


print(f"Test AUC: {roc_auc:.4f}")
print(f"Sensitivity (Recall): {sensitivity:.4f}")
print(f"Specificity: {specificity:.4f}")
```

Output :

```
Phase 5: Generating Final Test Reports...
Confusion Matrix:
[[ 92   0]
 [  7 118]]
```




```
Classification Report:
              precision    recall  f1-score   support

     Healthy       0.93      1.00      0.96        92
         MDD       1.00      0.94      0.97       125

    accuracy                           0.97       217
   macro avg       0.96      0.97      0.97       217
weighted avg       0.97      0.97      0.97       217

Test AUC: 0.9870
Sensitivity (Recall): 0.9440
Specificity: 1.0000
```

CHAPTER 7 : SYSTEM TESTING

7.1 Introduction

This chapter presents the testing process of the proposed AI-assisted depression detection system. After implementing the frontend interface, FastAPI backend, multimodal deep learning model, and Grad-CAM-style explainability support, system testing was conducted to verify that all major components work correctly and that the system produces reliable outputs.

The purpose of system testing is to ensure that the implemented prototype satisfies the functional and non-functional requirements defined in the previous chapters. The testing process covers the user interface, backend API, data input handling, model inference, diagnostic output, confidence score, heatmap visualization, and the full integration between the frontend and backend.

Since the system is intended as an academic research prototype and not a certified clinical diagnostic tool, the testing focuses on technical correctness, workflow reliability, and model performance on the independent test set.

7.2 Testing Objectives

The main objectives of system testing are:

1. To verify that the frontend interface allows the user to upload or select an MRI scan and enter clinical metadata.
2. To ensure that the FastAPI backend receives MRI-related input and clinical metadata correctly.
3. To confirm that the trained multimodal model can be loaded and used for inference.
4. To validate that the system returns the predicted class, confidence score, and heatmap output.
5. To test the connection between the frontend and backend using the public ngrok API URL.
6. To evaluate the model using independent test-set metrics such as accuracy, precision, recall, F1-score, sensitivity, specificity, confusion matrix, and ROC-AUC.
7. To confirm that the system displays a research-use disclaimer to avoid medical misinterpretation.

7.3 Testing Environment

The system was tested in the same development environment used during implementation.

Component	Testing Environment
Frontend	Next.js, React, TypeScript, Tailwind CSS
Backend	Python, FastAPI, Uvicorn
AI Model	PyTorch
Model File	best_multimodal_model.pth
Dataset Files	train_multimodal.csv, val_multimodal.csv, test_multimodal.csv
Testing Tool	Browser, FastAPI Swagger UI, ngrok
Hardware	Google Colab / GPU-supported environment

Table 7.1: System Testing Environment and Component Specifications

The backend was implemented using FastAPI and exposed through the `/api/analyze` endpoint. This endpoint receives MRI scan input and clinical metadata, prepares the data, runs inference, and returns the prediction result to the frontend. The implemented backend returns outputs such as diagnosis, confidence score, and heatmap data.

7.4 Testing Strategy

The testing strategy was divided into several levels:

7.4.1 Unit Testing

Unit testing was used to check individual components separately. This includes testing the MRI input handling, clinical metadata input, model loading function, prediction function, and output formatting.

7.4.2 Integration Testing

Integration testing was used to verify that the frontend and backend communicate correctly. When the user runs the analysis from the interface, the frontend sends the MRI scan and metadata to the FastAPI backend. The backend then processes the input, runs the model, and returns the result to the frontend.

7.4.3 System Testing

System testing was performed to test the complete workflow from start to finish. This includes opening the interface, uploading/selecting MRI input, entering metadata, running analysis, receiving prediction results, viewing the confidence score, and checking the heatmap visualization.

7.4.4 Model Performance Testing

Model performance testing was conducted using the independent test set. The notebook evaluates the trained model by loading `test_multimodal.csv`, rebuilding the same architecture, loading `best_multimodal_model.pth`, running inference without gradients, converting logits into probabilities, and calculating evaluation metrics such as confusion matrix, ROC curve, AUC, classification report, sensitivity, and specificity.

7.5 Test Data

The final dataset was divided into training, validation, and independent test sets. The independent test set was kept separate from the training and validation process to provide a fair estimate of model generalization.

Dataset Split	Number of Samples
Training set	1732
Validation set	217
Test set	217
Healthy-only training samples	736

Table 7.2: Summary of Training, Validation, and Independent Test Set Sizes

The system created four main CSV files:

- `train_multimodal.csv`
- `val_multimodal.csv`
- `test_multimodal.csv`
- `healthy_train_only.csv`

The independent test set was used only after model training was completed. This helps reduce data leakage and supports a more reliable evaluation of the final model.

7.6 Functional Testing

Functional testing was performed to ensure that each system requirement works as expected.

Test ID	Function Tested	Test Description	Expected Result	Status
FT1	MRI Input	Upload or select a T1-weighted MRI scan	MRI file is accepted and prepared for analysis	Passed
FT2	Clinical Metadata Input	Enter sex, age, education, and HAMD score	Metadata is accepted and sent to backend	Passed
FT3	AI Analysis	Run multimodal model inference	Model returns prediction and confidence score	Passed
FT4	Result Display	Display diagnostic result	Result appears as Healthy Control or MDD	Passed
FT5	Confidence Score	Display confidence percentage	Confidence score appears clearly in result screen	Passed

FT6	Heatmap Output	Display Grad-CAM-style output	Heatmap is returned and shown in interface	Passed
FT7	New Scan Workflow	Start a new analysis	Previous input and output are cleared	Passed
FT8	Disclaimer Display	Show research-use warning	Disclaimer appears on result screen	Passed

Table 7.3: Functional Testing Results and System Requirement Status

7.7 Backend API Testing

The backend API was tested to ensure that it receives requests correctly and returns the expected response format. The `/api/analyze` endpoint receives an uploaded MRI file and clinical metadata values, including sex, age, education, and HAMD score. After processing the request, the backend returns a JSON response containing the diagnosis, confidence score, and heatmap output.

A typical successful response is:

```
{
  "diagnosis": "Depression Signs Detected",
  "confidence": 87.3,
  "heatmap": "base64_encoded_heatmap"
}
```

The backend testing confirmed that:

1. The API endpoint can receive frontend requests.
2. Clinical metadata values are passed correctly.
3. The trained model file is loaded successfully.
4. The backend returns a structured JSON response.
5. The response can be displayed by the frontend.

7.8 Frontend Testing

Frontend testing was conducted using the web browser. The purpose was to verify that the user interface is clear, responsive, and follows the intended workflow.

The interface was tested across three main screens:

Upload Screen

The upload screen was tested to ensure that users can select or upload an MRI scan. The screen also explains the system workflow: upload, preprocess, analyze, and interpret.

Analysis Screen

The analysis screen was tested to ensure that clinical metadata fields are available and usable. The user can enter age, biological sex, education years, and HAMD score before running the analysis.

Result Screen

The result screen was tested to ensure that the diagnostic result, confidence score, MRI preview, and heatmap visualization appear correctly. The result screen also includes a disclaimer that the system is for research and decision-support purposes only, not a final clinical diagnosis.

7.9 Integration Testing

Integration testing verified the connection between the frontend and backend. During testing, the backend was exposed using ngrok so that the frontend could send requests to a public HTTPS API URL.

The ngrok public URL used during testing was:

`https://undepressive-esmeralda-frolicsomely.ngrok-free.dev`

ngrok allowed the locally running FastAPI backend to be tested through a public HTTPS URL without deploying the backend permanently to a cloud server. This helped test the communication between the frontend and backend during development.

The integration workflow was tested as follows:

1. User opens the frontend interface.
2. User uploads or selects an MRI scan.
3. User enters clinical metadata.

4. Frontend sends the request to the FastAPI backend.
5. Backend runs preprocessing and model inference.
6. Backend returns diagnosis, confidence score, and heatmap.
7. Frontend displays the returned results.

The integration test passed because the frontend was able to communicate with the backend and display the returned AI output.

7.10 Model Testing and Diagnostic Performance

The final trained multimodal model was tested using the independent test set. The evaluation was performed using the saved best model checkpoint, `best_multimodal_model.pth`.

The model testing process included:

1. Loading the independent test dataset.
2. Rebuilding the same multimodal model architecture.
3. Loading the best trained weights.
4. Running inference without gradient calculation.
5. Applying sigmoid activation to obtain prediction probabilities.
6. Applying a threshold of 0.5 to generate binary predictions.
7. Calculating confusion matrix, classification report, ROC-AUC, sensitivity, and specificity.

The independent test-set results were:

Metric	Result
Accuracy	97%
Test AUC	0.9870
Sensitivity / Recall	0.9440
Specificity	1.0000

Table 7.4: Diagnostic Performance Metrics on the Independent Test Set

The confusion matrix was:

	Predicted Healthy	**Predicted MDD**
Actual Healthy	92	0
Actual MDD	7	118

Table 7.5: Confusion Matrix for Predicted vs. Actual Clinical Classes

The classification report showed that the model achieved 0.93 precision for Healthy Control and 1.00 precision for MDD. The overall accuracy was 0.97 on 217 independent test samples.

These results indicate that the model performed strongly on the independent test set. However, because the system is an academic prototype, these results should be interpreted as research performance only and not as clinical certification.

7.11 Confusion Matrix Interpretation

The confusion matrix provides a detailed view of the model's classification behavior.

The model correctly classified 92 Healthy Control subjects as Healthy and 118 MDD subjects as MDD. There were no false positives, meaning no Healthy Control subject was incorrectly classified as MDD. However, 7 MDD subjects were classified as Healthy Control.

In medical AI systems, false negatives are important because they represent cases where the model fails to detect actual MDD. Therefore, even though the overall performance is strong, reducing false negatives should be a priority in future improvement.

7.12 ROC-AUC Testing

The ROC-AUC value was used to measure the model's ability to distinguish between MDD and Healthy Control subjects across different classification thresholds. The model achieved a Test AUC of 0.9870, which indicates strong discrimination between the two classes.

A higher AUC means the model is better at ranking MDD subjects above Healthy Control subjects based on predicted probability. This supports the reliability of the model as a research decision-support system.

7.13 Explainability Testing

Explainability testing focused on verifying that the system can return and display Grad-CAM-style heatmap outputs. The purpose of this feature is to help users understand which MRI regions contributed to the model's prediction.

The implemented backend generates the heatmap during inference and returns it to the frontend. The interface can then display the heatmap over MRI slices so that users can visually inspect the highlighted regions.

The explainability test checked that:

1. The backend can generate a heatmap output.
2. The heatmap is returned in the API response.
3. The frontend can display the heatmap.
4. The user can compare the heatmap with the MRI preview.
5. The output is presented with a research-use disclaimer.

This test supports the transparency requirement of the system, especially because deep learning models can be difficult to interpret.

7.14 Non-Functional Testing

Non-functional testing was conducted to evaluate usability, reliability, maintainability, and ethical presentation.

Requirement	Test Description	Result
Usability	Interface screens are clear and easy to follow	Passed
Reliability	System returns prediction output without workflow interruption	Passed
Maintainability	Frontend, backend, and model are separated into modular components	Passed

Performance	Backend can run inference using the trained model	Passed
Security & Privacy	System avoids displaying personal identifiers in the result screen	Passed
Ethical Use	Disclaimer states that the system is not a final medical diagnosis	Passed

Table 7.6: Non-Functional Testing Results and Quality Evaluation

The testing confirmed that the system is suitable as an academic prototype for demonstrating AI-assisted depression detection from brain MRI.

7.15 Testing Limitations

Although the system passed the main testing stages, several limitations remain.

First, the system was tested as a research prototype, not as a certified clinical product. Therefore, its predictions should not be used as final medical decisions.

Second, the model was evaluated using the available independent test set, but additional external validation on new datasets would be needed to confirm generalization across hospitals, scanners, and populations.

Third, the interface and backend were tested through ngrok during development. A permanent production deployment would require stronger security, authentication, storage management, and server monitoring.

Fourth, although Grad-CAM-style heatmaps improve interpretability, they should be reviewed carefully by domain experts to confirm whether the highlighted regions are clinically meaningful.

7.16 Chapter Summary

This chapter presented the testing process of the AI-assisted depression detection system. The testing covered functional testing, backend API testing, frontend testing, integration testing, model performance testing, explainability testing, and non-functional testing.

The backend was tested through the FastAPI `/api/analyze` endpoint, which receives MRI input and clinical metadata, runs the trained multimodal model, and returns the diagnostic output, confidence score, and heatmap data. The frontend was tested to ensure that users can upload MRI data, enter clinical metadata, run analysis, and view the result clearly.

The model was evaluated on an independent test set of 217 samples and achieved 97% accuracy, 0.9870 AUC, 0.9440 sensitivity, and 1.0000 specificity. These results show strong performance for the academic prototype. However, the system remains a research decision-support tool and should not be interpreted as a certified clinical diagnostic system.

Overall, the testing phase confirmed that the implemented system meets the main functional requirements and successfully demonstrates an end-to-end workflow for multimodal MDD detection using 3D brain MRI and clinical metadata.

CHAPTER 8 : FUTURE WORK AND CONCLUSION

8.1 Introduction

This chapter presents the future work and final conclusion of the proposed AI-assisted Major Depressive Disorder detection system using brain MRI and clinical metadata. Previous chapters described the system design, implementation, model development, and testing process. The project successfully developed a research prototype that combines a web-based interface, FastAPI backend, multimodal deep learning model, and Grad-CAM-style explainability support.

The system demonstrated the ability to process 3D structural MRI data together with clinical metadata and produce a binary classification result between Healthy Control and Major Depressive Disorder. The final model was evaluated using an independent test set and achieved strong diagnostic performance, including 97% accuracy, 0.9870 AUC, 0.9440 sensitivity, and 1.0000 specificity. These results show that the proposed system is promising as an academic research prototype and decision-support framework.

However, despite the positive results, the system is not a certified clinical diagnostic tool. Further development, validation, and clinical testing are required before such a system can be used in real medical environments. Therefore, this chapter discusses possible future improvements and concludes the overall project.

8.2 Summary of Project Achievements

The project achieved its main goal by developing an AI-assisted framework for depression detection from brain MRI. The system was designed to support objective and data-driven analysis of Major Depressive Disorder using structural MRI scans and clinical metadata.

The main achievements of the project include:

1. Development of a multimodal dataset pipeline that combines MRI volume paths, diagnostic labels, and clinical metadata.
2. Implementation of a custom PyTorch dataset engine for loading 3D MRI volumes and clinical features.
3. Training of a 3D convolutional autoencoder using Healthy Control MRI scans to learn a normative anatomical representation.
4. Development of a multimodal late-fusion model combining:
    - 3D MRI imaging branch
    - Squeeze-and-Excitation attention block
    - Clinical metadata MLP branch

    - Binary classification head
5. Implementation of a web-based frontend interface using Next.js, React, TypeScript, and Tailwind CSS.
6. Implementation of a FastAPI backend for receiving MRI-related inputs, running inference, and returning results.
7. Integration of prediction output, confidence score, and Grad-CAM-style heatmap visualization.
8. Testing of the full system workflow, including frontend, backend, model inference, API response, and result display.
9. Evaluation of the final model on an independent test set.

The system therefore demonstrates an end-to-end workflow for AI-assisted MDD detection, starting from MRI and clinical data preparation and ending with prediction visualization in an interactive prototype.

8.3 Future Work

Although the implemented system successfully meets the main objectives of the project, several improvements can be made in future versions. These improvements would help increase reliability, clinical usefulness, interpretability, and deployment readiness.

8.3.1 External Dataset Validation

The current model was evaluated using the available independent test set. Although the results were strong, future work should validate the model on additional external datasets collected from different hospitals, scanners, and populations.

External validation is important because medical AI models may perform well on one dataset but show lower performance when tested on data from another source. Differences in scanner type, acquisition protocol, population demographics, and clinical labeling can affect model generalization.

Therefore, future work should test the model on new unseen datasets to confirm that it can generalize beyond the current experimental setting.

8.3.2 Local Saudi Dataset Collection

One important future direction is to collect and validate the system using local Saudi clinical MRI data. This would make the system more relevant to the Saudi healthcare context and support national digital health research.

Using local datasets would help examine whether the model performs well on Saudi patients and clinical environments. It would also support the development of AI-

assisted mental health tools aligned with Saudi Vision 2030 healthcare transformation goals.

8.3.3 Improve Grad-CAM Explainability

The current system supports Grad-CAM-style heatmap visualization to highlight MRI regions that contribute to the model's prediction. In future work, this explainability component can be improved by validating the highlighted regions with radiologists, psychiatrists, or neuroscience experts.

Future improvements may include:

- More accurate 3D Grad-CAM visualization.
- Slice-by-slice heatmap navigation.
- Region-based explanation using anatomical brain atlases.
- Comparison between heatmap regions and known MDD biomarkers such as the hippocampus, prefrontal cortex, and anterior cingulate cortex.
- Explainability for the clinical metadata branch.

This would make the model more transparent and clinically trustworthy.

8.3.4 Reduce False Negatives

The confusion matrix showed that the model correctly classified most cases, but there were still 7 MDD subjects classified as Healthy Control. In medical AI systems, false negatives are important because they represent actual patients that the model failed to detect.

Future work should focus on reducing false negatives by improving sensitivity. This can be done through:

- Threshold tuning instead of using a fixed 0.5 threshold.
- Cost-sensitive learning.
- More balanced training strategies.
- Additional clinical features.
- Larger and more diverse training data.

Reducing false negatives would make the system safer as a decision-support tool.

8.3.5 Improve Backend Deployment

The current backend was tested using FastAPI and ngrok during development. While this is suitable for testing, a production-ready system would require a more secure and stable deployment environment.

Future work should deploy the backend on a cloud server or hospital server with:

- HTTPS security.
- Authentication and user access control.
- Secure file upload handling.
- Automatic deletion of temporary MRI files.
- Server monitoring.
- Error logging.
- Scalable GPU inference support.

This would make the system more reliable for real-world testing.

8.3.6 Improve Data Privacy and Security

Since MRI scans and clinical metadata are sensitive medical data, future work should strengthen privacy and security measures. The system should ensure that uploaded files are not stored permanently unless required, and that no personal identifiers are displayed in the interface.

Future improvements may include:

- Data anonymization.
- Secure encrypted storage.
- Role-based access control.
- Audit logs.
- Compliance with medical data privacy standards.
- Clear consent and research-use policies.

These improvements are necessary before the system can be tested in real clinical environments.

8.3.7 Add More Clinical and Multimodal Features

The current system uses MRI data and four clinical variables: sex, age, education, and HAMD score. In future work, the system can be expanded by adding more clinical or multimodal features.

Possible additional features include:

- PHQ-9 score.
- Medication history.
- Depression duration.
- Family history.
- Cognitive assessment scores.
- Resting-state fMRI features.
- EEG data.
- Speech or behavioral features.

Adding more clinically relevant features may improve the model's diagnostic performance and make the prediction more comprehensive.

8.3.8 Federated Learning

Federated learning is another important future direction. Instead of collecting all hospital data in one central location, federated learning allows the model to be trained across multiple hospitals without moving patient data outside each institution.

This approach would improve privacy and allow the model to learn from larger and more diverse datasets. It would be especially useful for medical AI applications where patient data cannot be easily shared due to privacy and ethical restrictions.

8.3.9 Clinical Trial and Expert Evaluation

Before the system can be considered for real clinical use, it must be evaluated by medical experts. Future work should involve psychiatrists, radiologists, and clinical researchers to assess whether the system outputs are useful, understandable, and clinically meaningful.

Expert evaluation should include:

- Reviewing prediction outputs.
- Reviewing Grad-CAM heatmaps.
- Comparing AI predictions with clinical diagnosis.
- Identifying cases where the model makes mistakes.
- Assessing whether the interface is practical for clinical workflow.

This step is necessary to move from an academic prototype toward a clinically useful decision-support system.

8.3.10 Mobile and Cloud-Based Version

The current interface is web-based. Future work may extend the system into a cloud-based platform or mobile-compatible dashboard. This would make the prototype easier to access for researchers, clinicians, and academic evaluators.

A cloud-based version could allow users to upload MRI scans, run analysis, and view results from different locations, while still maintaining strict privacy and security controls.

8.4 Project Limitations

Although the project achieved its main objectives, several limitations remain.

First, the system is a research prototype and not a certified medical diagnostic tool. The output should be interpreted only as decision-support information and not as a final clinical diagnosis.

Second, the model was evaluated using the available dataset and independent test split, but further external validation is required to confirm generalization across different hospitals, scanners, and populations.

Third, MRI-based depression detection is a complex task because MDD is influenced by biological, psychological, social, and clinical factors. Structural MRI alone may not capture the full complexity of depression.

Fourth, although Grad-CAM improves interpretability, heatmaps still require expert review to confirm that the highlighted regions are clinically meaningful.

Finally, the backend deployment through ngrok is suitable for development and testing, but not enough for production-level clinical use.

8.5 Final Conclusion

This project presented the design, development, implementation, and testing of an AI-assisted system for detecting Major Depressive Disorder from brain MRI and clinical metadata. The proposed system used a SOTA-inspired multimodal deep learning approach that combines 3D structural MRI features with clinical variables through a late-fusion architecture.

The model development included a 3D convolutional autoencoder trained on Healthy Control MRI scans to learn a normative representation of healthy brain anatomy. The learned encoder was then used as an anatomical blueprint for the final diagnostic model. The final model combined a 3D CNN imaging branch, Squeeze-and-Excitation attention, a clinical metadata MLP branch, and a binary classification head.

The system was implemented as a working prototype using a Next.js and React frontend, a FastAPI backend, and a PyTorch-based deep learning model. The interface allows users to upload or select MRI input, enter clinical metadata, run AI analysis, and view the prediction result, confidence score, MRI preview, and Grad-CAM-style heatmap output.

Testing confirmed that the system satisfies the main functional requirements. The independent test-set evaluation showed strong performance, with 97% accuracy, 0.9870 AUC, 0.9440 sensitivity, and 1.0000 specificity. These results demonstrate that

the proposed framework is promising as an academic research prototype for AI-assisted depression detection.

Overall, this project contributes to the field of AI-based mental health research by demonstrating how 3D brain MRI, clinical metadata, deep learning, and explainability can be combined into a single decision-support framework. While the system is not ready for clinical diagnosis, it provides a strong foundation for future research, external validation, clinical expert review, and possible development toward secure real-world medical AI applications.

8.6 Chapter Summary

This chapter presented the future work and final conclusion of the project. It summarized the main achievements of the AI-assisted MDD detection system and discussed future directions such as external validation, local Saudi dataset collection, improved explainability, false negative reduction, secure deployment, privacy protection, additional multimodal features, federated learning, and clinical expert evaluation.

The chapter concluded that the project successfully developed and tested a multimodal AI-assisted prototype for depression detection from 3D brain MRI and clinical metadata. The system achieved strong research-level performance and provides a solid foundation for future improvements toward clinically reliable and explainable AI-based mental health support.

REFERENCES

[1] World Health Organization, "Depressive disorder (depression)," *WHO Fact Sheets*, Aug. 29, 2025. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/depression

[2] M. Mousavian, J. Chen, Z. Traylor, and S. Greening, "Depression detection from sMRI and rs-fMRI images using machine learning," *Journal of Intelligent Information Systems*, vol. 57, no. 2, pp. 395–418, 2021, doi: 10.1007/s10844-021-00653-w.

[3] R. Li, Y. Huang, Y. Li, and X. Lai, "MRI-based deep learning for differentiating between bipolar and major depressive disorders," *Psychiatry Research: Neuroimaging*, vol. 345, Art. no. 111907, 2024, doi: 10.1016/j.pscychresns.2024.111907.

[4] M. Squires, X. Tao, S. Elangovan, R. Gururajan, X. Zhou, U. R. Acharya, and Y. Li, "Deep learning and machine learning in psychiatry: A survey of current progress in depression detection, diagnosis and treatment," *Brain Informatics*, vol. 10, no. 1, Art. no. 10, 2023, doi: 10.1186/s40708-023-00188-6.

[5] Vandana et al., "Federated learning and deep learning framework for MRI image and speech signal-based multi-modal depression detection," *Computational Biology and Chemistry*, vol. 113, Art. no. 108232, 2024, doi: 10.1016/j.compbiolchem.2024.108232.

[6] Q. Li, X. Liu, X. Hu, M. R. Ahad, M. Ren, L. Yao, and Y. Huang, "Machine learning-based prediction of depressive disorders via various data modalities: A survey," *IEEE/CAA Journal of Automatica Sinica*, vol. 12, no. 7, pp. 1320–1349, Jul. 2025, doi: 10.1109/JAS.2025.125393.

[7] Y. Chen, W. Zhao, S. Yi, and J. Liu, "The diagnostic performance of machine learning based on resting-state functional magnetic resonance imaging data for major depressive disorders: A systematic review and meta-analysis," *Frontiers in Neuroscience*, vol. 17, Art. no. 1174080, 2023, doi: 10.3389/fnins.2023.1174080.

[8] M. Mansoor and K. Ansari, "Advancing early detection of Major Depressive Disorder using multisite functional magnetic resonance imaging data: Comparative analysis of AI models," *JMIRx Med*, vol. 6, Art. no. e65417, 2025, doi: 10.2196/65417.

[9] R-fMRI Lab, "Data Sharing of the REST-meta-MDD Project from the DIRECT Consortium," 2022. [Online]. Available: https://rfmri.org/REST-meta-MDD

[10] R-fMRI Lab, "Data Sharing of the Phase II from the DIRECT Consortium," 2025. [Online]. Available: https://rfmri.org/DIRECTPhase2

[11] Saudi Vision 2030, "Health Sector Transformation Program," 2024. [Online]. Available: https://www.vision2030.gov.sa

[12] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.

[13] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[14] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond Empirical Risk Minimization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[15] R. R. Selvaraju et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.